



The Open University

M263 Building Blocks of Software

Unit 8

Binary trees

Study guide

The three sections of this unit are of about equal length and importance. There are WorkPad activities associated with each of the three sections. You should not spend more than about 15% of your study time for this unit on practical work.

The unit introduces a new form of data structure (a binary tree), some associated concepts and terminology, and some special forms of this data structure used in particular applications. A class (**BinTree of X**) that has a binary tree as its form of data is introduced. Simple code and implementations using **BinTree of X** are discussed, but implementations are not given for the more complicated processes considered here. (Recursion, discussed in Unit 13, is an important and convenient way of designing implementations on binary trees.) The main thrust of this unit is to familiarise you with the idea of a binary tree, and processes involving binary trees, rather than with code involving the class **BinTree**. (In this, it is similar to Unit 7 on sets.)

This publication forms part of an Open University course. Details of this and other Open University courses can be obtained from the Course Information and Advice Centre, PO Box 724, The Open University, Milton Keynes, MK7 6ZS, United Kingdom: tel. +44 (0)1908 653231, e-mail general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by the Open University.

To purchase a selection of Open University course materials, please visit the Open University webshop at www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858785; fax +44 (0)1908 858787; e-mail ouwenq@open.ac.uk

The Open University, Walton Hall, Milton Keynes, MK7 6AA.

First published 2004. Second edition 2008.

Copyright © 2008 The Open University

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd of 90 Tottenham Court Road, London W1T 4LP.

Edited, designed and typeset by The Open University, using the Open University T_EX System.

Printed and bound in the United Kingdom by The Charlesworth Group, Wakefield.

ISBN 978 07492 2301 4

Contents

Introduction	4
1 Binary trees	5
1.1 Binary tree diagrams	6
1.2 The class BinTree of X	12
2 Binary trees and vectors	20
2.1 Complete trees	20
2.2 Tree traversals	24
2.3 Growing trees by division	27
3 Sorted binary trees	32
3.1 Binary search trees	34
3.2 Heap trees	39
Summary of Unit 8	45
Solutions to the exercises	46
Index	48

Introduction

In Unit 6 we discussed two collection classes, **Vector** and **mString**, whose underlying form of data is a sequence. We show the state of an object of type **Vector** as in the example below.

[11, 22, 33]

The elements of a vector are indexed, with the first element (at index 1) shown on the left. We think of an object of type **mString** in a similar way, as an indexed sequence of elements; in a string, all the elements are characters. So the way we envisage an object of type **Vector** or **mString** is linear (or one-dimensional). That is, we think of the elements in the collection arranged in a single line, in an order corresponding to the indexing of the elements.

In this unit we consider an important form of collection that is not linear, called a **tree**. Tree structures are extensively used in computing. A tree structure stores a collection in a way that is most naturally shown graphically — examples are given in Figures I1 and I2. In a tree structure there is a branching relationship between the positions at which elements are stored.

The diagrams that represent tree structures are two-dimensional. You have probably seen examples of trees, since such diagrams are frequently used to present information of a suitable form.

Figure I1 shows a graphical representation of the way folders are stored in a computer.

In Figure I1, we see that the folder M263 contains a folder WorkPad that contains a folder jre. The folder jre contains two folders bin and lib, and each of these contain further folders; bin containing two further folders, and so on.

In this case, the graphic not only shows the hierarchy of folders but is also part of the user interface. A user can “point and click” on the graphic to open a particular folder. In more primitive operating systems, in order to access any particular folder we would need to know both the name of that folder and the correct path from the folder M263 to the particular folder lower down in the tree. For instance, to access the folder **cursors**, the path followed would be given as M263\WorkPad\jre\lib\images\cursors.

Figure I1 shows the most general form that a tree may have. The way the tree is displayed in Figure I1 is convenient for large trees, but it does not show the characteristic structure of a tree quite as clearly as it might. This is more easily seen in Figure I2. The tree there shows (part of) the pedigree of the thoroughbred horse Lammtarra. Such a pedigree shows the ancestors of a given horse.

In Figure I2, the parents of each horse are shown below it, linked to it by lines. The mother is on the left and the father on the right. So, for example, Snow Bride is the mother of Lammtarra, while Blushing Groom is the father of Snow Bride.

We would not regard a set as one-dimensional, since the elements of a set are not indexed, so that there is no order in which the elements of a set should be written.

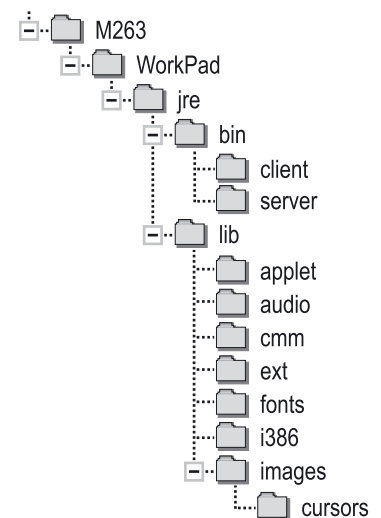


Figure I1

In Microsoft Windows XP, this view of the file structure can be seen by looking in **My Computer** and clicking **Folders**. Older versions of Windows show file structures this way as a default.

Such diagrams are of great interest to breeders of pedigree animals.

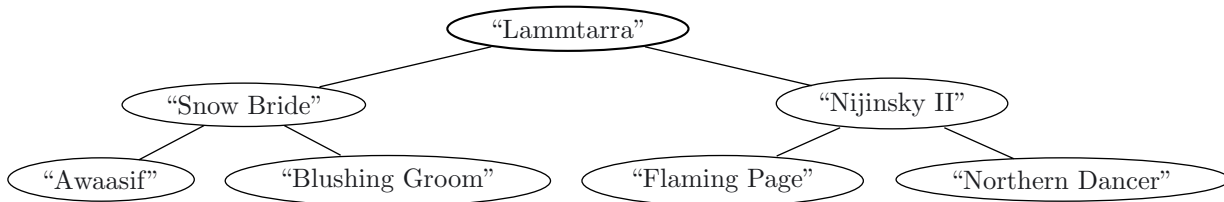


Figure I2

In a tree, a position in the structure where an item is stored is called a **node**. The nodes are connected by **branches** (shown by lines in Figure I2) in a hierarchy. There is one node (called the **root**) that has no branch leading in to it. In Figure I2 the root is “Lammtarra”. In Figure I1, some nodes have one branch leading away from them and others have more than one branch. For instance, there is just one branch from M263 to the position of the folder **WorkPad**, but seven branches lead from the position of the folder **lib** to lower folders. However, Figure I1 shows a very general form of tree, and in this unit we shall concentrate on a particular form, which is illustrated in Figure I2. This is called a **binary tree**. In a binary tree, there are at most two branches leading out of any node, and these are identified as a **left** branch and a **right** branch. (In Figure I2, a left branch leads to a mother while a right branch leads to a father.)

In a sequence, the same data item may appear more than once. The same is true of a tree. For example, it would not be unusual to see the same male horse appearing at more than one position in the tree showing a pedigree.

As illustrated in Figure I1, not all trees are binary trees, but in this unit, we shall concentrate on binary trees. In Section 1, we look at fundamental ideas associated with binary trees, and introduce a class that has binary trees as its form of data. In Section 2, we look at processes that move data between sequence-based collections and binary trees. Forms of data such as integers and characters have comparison operations. In Unit 6, we used this to sort data items within a sequence. In a similar way, data items can be placed within a binary tree in a way that is consistent with the size of the items. We will call such trees “sorted”. There is more than one way in which a tree may be sorted, and we consider two approaches in Section 3: **binary search** trees and **heap** trees.

1 Binary trees

Figures I1 and I2 display the information they contain in a diagrammatic way. It is the diagram of a binary tree that best represents the idea of this form of collection, and it is through the diagram that we will introduce the terminology and explain how the methods of the class **BinTree** are used to manipulate the information contained in the collection.

A tree diagram uses a graphical display to convey information about the data it holds. This visual abstraction shows how the data is connected and the paths we can take through the data. Here, we shall not be concerned with how a computer stores data in such a tree structure. We will be concerned with how we can manipulate data stored in such collections: how we can access the stored data, or rearrange such collections. The best way to think

about a binary tree structure is as a diagram. We shall show such diagrams in a similar way to Figure I2. Another example is given in Figure 1.1. This shows a collection of integers. The lines indicate ways of proceeding through the collection. We will start by using such diagrams to discuss the way such a tree is structured. We will then go on to introduce a class **BinTree of X**, whose form of data is a tree storing items of type **X**.

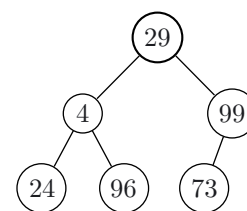


Figure 1.1

1.1 Binary tree diagrams

A binary tree consists of a number of **nodes** (shown as circles in our diagrams). These are connected by **branches** (shown by lines in the diagrams). The binary trees we consider will hold data in each of the nodes (in the diagram, this data is shown inside the circles representing the nodes). However, a binary tree has an underlying structure that is separate from the data being stored, and we will start by looking at this.

At the top of a tree diagram there is a single node, called the **root**. It is conventional to draw trees with the “root” at the top, and the branches growing down. There is one special case, the **empty tree**, which has no nodes at all. The empty tree has no root, but all other trees do. In a binary tree, there are at most two branches leading down from any node. These are a **left branch** and a **right branch** (see Figure 1.2). Below a node in a binary tree there may be two, one, or no branches, as is exemplified in Figure 1.1. If there is only one branch below a node we must identify that as either a left or right branch. In Figure 1.1, below the node storing 99 there is a left branch but no right branch. When drawing a binary tree in which there is a single branch below a node, it is important to show clearly whether that is a left or a right branch!

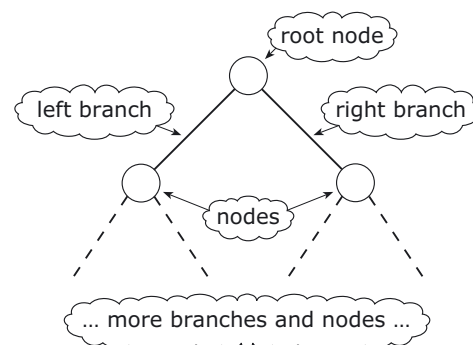


Figure 1.2

Figure 1.3 illustrates further terminology that we use in the context of binary trees. The root node is at the top of the tree. It is the only node that does not have a branch leading in to it from above. At the bottom of the tree are nodes with no branches leading out below them. Such nodes are called **leaves**, or **leaf nodes**. A node that is neither the root nor a leaf node is called an **interior node**. If a node (say P) has branches below it, we refer to the nodes at the end of these branches as **children** of P, and to P as the **parent** of these nodes. (So parent/child is a relationship between two nodes in a tree, saying that there is a branch leading down from the parent to the child.)

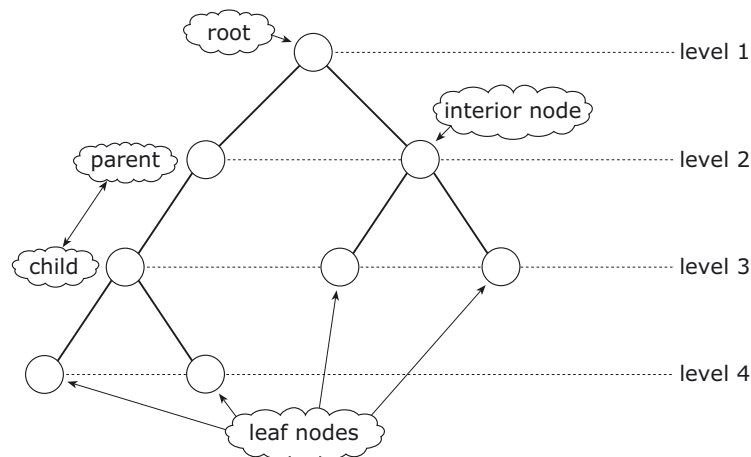


Figure 1.3

At any node in a tree other than the root, there is exactly one branch leading into the node from above. This means that there is a unique path along branches from the root node to any particular node. In the file storage system illustrated in Figure 1.1, this property of trees gives the unique path identifying each folder.

We arrange the nodes of a tree in **levels**, depending on the number of branches one must follow to return to the root. The root itself is at level 1. The children of the root node are at level 2. The children of a node at level 2 are at level 3, and so on. The **depth** of a tree is the number of levels that it has. An empty tree has depth 0, while a tree consisting of just a root node has depth 1. (In general, the depth is 1 more than the greatest number of branches that one can follow by starting at the root and working downwards until one reaches a leaf.)

Suppose we have a binary tree of depth n . We can make the following observations. The path from the root to any particular node visits at most n nodes (including the root and the final node). Since each node can have no more than two children, there can be at most:

- 1 node at level 1;
- 2 nodes at level 2;
- 4 nodes at level 3,

and so on. In general, there can be (at most) twice as many nodes at each level as there are in the level above. So at level i , the maximum possible number of nodes is 2^{i-1} .

In a binary tree, a node can have at most two children. These will appear at the end of either the left or right branch below the node, and so may be referred to as the **left child** or the **right child**. The idea of left- and right-handedness is very important within the structure of a binary tree. In Figure 1.4, we look at the binary tree shown in Figure 1.3 in a different way. If we follow the left branch below the root node, this leads to a part of the tree that is shaded in Figure 1.4(a). If we look at this part of the tree, we can see that it is a tree itself. This has as its root the left child of the root in the original tree. We call this the **left subtree** of the original tree. Similarly, the right branch below the root leads to a part of the original tree that is again a tree — its **right subtree**. Except for the empty tree, any tree has left and right subtrees (which may be empty trees). As we will see later in this section, the subtrees are an important feature of a binary tree.

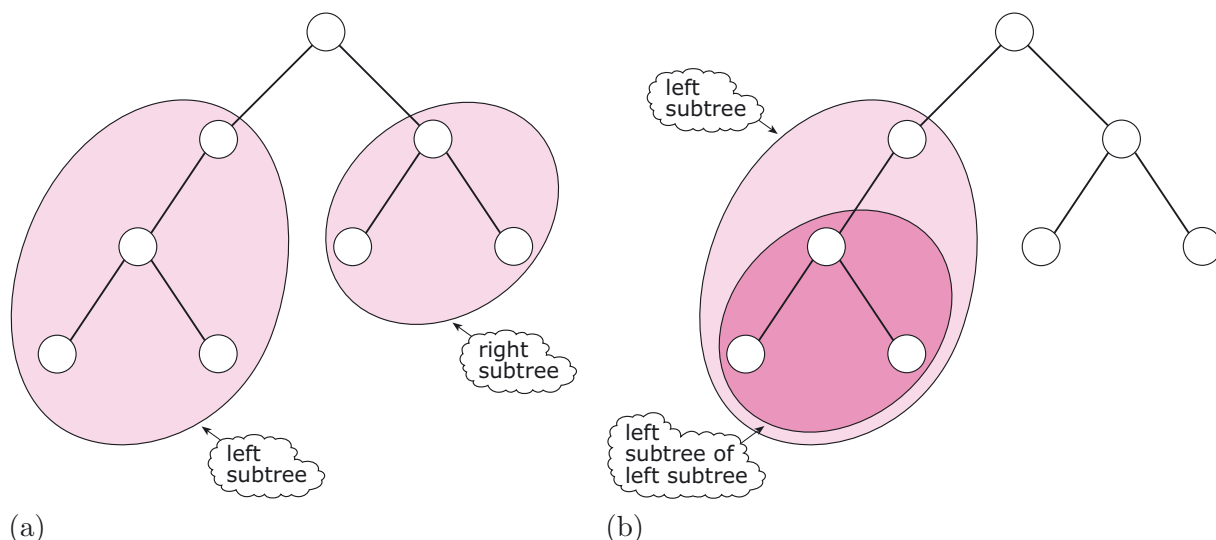


Figure 1.4

In Figure 1.4(b), we focus on the left subtree of the tree in Figure 1.4(a). That is a (non-empty) tree, and itself has right and left subtrees. Its right subtree is empty, while its left subtree is highlighted in the diagram.

It can be useful to label locations in a tree, since the same data may be stored at different locations. In a diagram such as Figure 1.5, we need to be clear that the letters are being used to label locations in the tree, and are not stored data items. Language such as “child”, “parent” and “leaf” relates to locations in the tree structure. If we choose any node in a tree, and look at the part of the tree consisting of that node and everything below it, then this will form a tree with the chosen node as its root. For example, in Figure 1.5, the node F is the root of the tree containing the nodes F, H and I. Any such tree can be referred to as a **subtree** of the original tree. However, the terminology subtree is most often used for the right and left subtrees. (In Figure 1.5, the left subtree has root B and the right subtree has root C.)

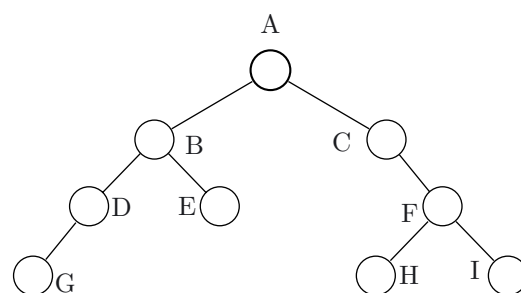


Figure 1.5

Activity 1.1

Figure 1.5 shows a binary tree, t , in which each node is identified by a letter.

- Identify the following parts of this binary tree t .
 - The children of node D.
 - The parent of node F.
 - The leaf nodes of t .
 - The root node of t .
 - The left subtree of t .
- What is the depth of each of the subtrees of the right subtree of t ?

Discussion

- The node D only has one child (a left child), which is the node G.
 - The parent of node F is the node C.
 - The leaves are the nodes G, E, H and I.
 - The root node is A.
 - The left subtree of the tree is the binary tree with root node B, and is shown in Figure 1.6.
- The right subtree is the binary tree with root node C. This has an empty left subtree, so this has depth 0. The right subtree of the right subtree has root node F, and this has depth 2.

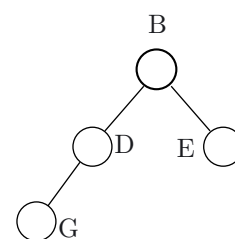


Figure 1.6

Setting aside the data that may be stored, there are a limited number of possible structures that a tree may have. The only tree of depth 0 is the empty tree. We shall write this as $ET()$, for reasons that we shall discuss later in the section.

A tree of depth 1 must consist of a single node, the root, and we shall show such trees in a diagram of the form shown in Figure 1.7.



Figure 1.7

For the remainder of this unit, we will usually shorten “binary tree” to “tree”.

Activity 1.2

For a binary tree of depth 2, draw diagrams illustrating all the possible forms that its structure may have.

Discussion The three possible trees are shown in Figure 1.8. We must have a root node, and this may have: just a left child; just a right child; or two children.

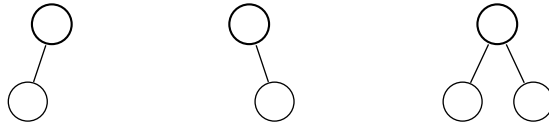


Figure 1.8 (a) (b) (c)

We have been concentrating on the structure of a tree, and drawing diagrams showing only the structure (the nodes and branches). We now return our attention to trees in which data is stored. Any form of data may be stored, but in our examples we shall only consider trees that contain elements of the types **Int**, **Char** or **mString**. As with other collection structures, we shall only store data of a consistent type within a particular tree. So we may have a tree storing integers, or a tree storing characters, but shall not consider trees storing a mixture of integers and characters.

A key feature of the trees that we shall consider in this unit is that all nodes must store something. A tree can have no “empty nodes”.

Another feature of any tree is that all nodes (except the root) must have a branch leading to them from above. So the diagrams in Figure 1.9 show collections that are *not* diagrams of binary trees. Figure 1.9(a) shows a single empty node. This is *not* a diagram for the empty tree. Figure 1.9(b) also has an empty node, while Figure 1.9(c) shows a node (that storing 33) that has no branch leading into it from above.

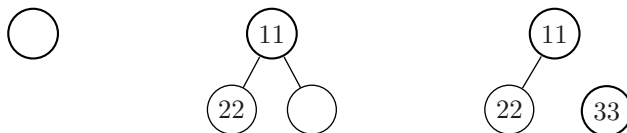


Figure 1.9 (a) (b) (c)

It is important to distinguish between a tree of depth 1 that has an element stored at its single node and that element itself. Figure 1.10 shows a tree, consisting of a single node at which the integer 11 is stored. This tree is quite different from the integer 11.



Figure 1.10

We noted in Activity 1.2 that there are three distinct structures for a tree of depth 2. In particular, the trees in parts (a) and (b) of Figure 1.8 are not the same. Each has only one leaf, but one has a left branch and the other has a right branch. It is important to observe that these two binary trees do not have the same structure. For example, the two trees shown in Figure 1.11 are not the same, even though they store the same two integers.

An essential property of a binary tree is that it consists of a root node and left and right subtrees, which are themselves binary trees. The only exception to this is the empty tree. This leads to the following description of a binary tree.

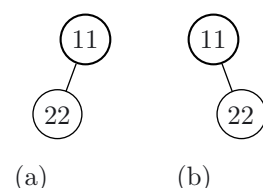


Figure 1.11

This is not always the case in general tree diagrams. Some types of trees only store elements at their leaf nodes.

A **binary tree** consists of either:

- no nodes (i.e. is an empty tree); or
- a root node and two binary trees (that are its left subtree and its right subtree).

This suggests the following procedure for drawing a binary tree.

Write down its root node. Then add in the left subtree and the right subtree (using the same procedure).

Described like this, the procedure would never stop. In practice we would stop, of course, and this happens when all the trees remaining to be added are empty. (We do not show empty subtrees in our diagrams, so there are then no more nodes to be added.)

Notice that, in the description boxed above, we are expressing the structure of a binary tree in terms of other binary trees. (To paraphrase Gertrude Stein, a “Tree is a tree is a tree is a tree”). This type of description is said to be **recursive**, and the subject of recursion will be introduced and discussed in Unit 13.

“Rose is a rose is a rose is a rose.”, from *Sacred Emily* by Gertrude Stein (1874–1946).

Consider the tree of integers shown in Figure 1.11(a). This has the integer 11 at its root node. For convenience, we will say the “the tree has root 11”, rather than “the value at the root node of the tree is 11”. The tree in Figure 1.11(a) has a left subtree and a right subtree. The right subtree is empty. The left subtree has root 22, while its left and right subtrees are empty.

Remember, empty trees are not shown on the diagram.

The graphical form of a binary tree conveys information in a way that is quite easy to interpret. However, the diagrams do take up space, and there are times when a more compact notation is useful. We will use a compact, non-graphical notation for trees occasionally, and we introduce this now. The basic idea is that we can write any binary tree in the following form.

(root left subtree right subtree)

We need to start with a notation for an empty tree. In this non-graphical notation, we write an empty tree as a pair of brackets.

()

Note that there are no commas, or any other separators here. The brackets will act as separators.

The tree in Figure 1.10 (with empty left and right subtrees, and just a root node of 11), would be written as follows.

(11 () ())

The tree in Figure 1.11(a) is expressed as follows.

(11 (22 () ()) ())

The tree in Figure 1.11(b) is written as follows.

(11 () (22 () ()))

As the tree gets larger this notation becomes harder to interpret, with a proliferation of brackets—as you can no doubt imagine. We shall refer to this as **linear notation** for writing a tree. This linear notation is much less easy to interpret than the graphical representation of a tree, and we shall use it only rarely.

Activity 1.3

For the binary tree shown in Figure 1.12, write down the linear notation for each of: its left subtree; its right subtree; the whole tree.

Discussion The left subtree has: root node 23, left subtree empty; right subtree the single node 99. So in linear notation, this is as below.

(23 () (99 () ()))

The right subtree has just the root node 35, so in linear notation, this is

(35 () ()).

For the entire tree, we can use these results. We must start with its root node, 17. We get the linear notation below.

(17 (23 () (99 () ())) (35 () ()))

(This is: open bracket; root node; linear notation for the left subtree; linear notation for the right subtree; close bracket.)

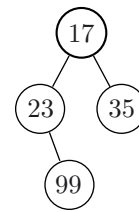


Figure 1.12

EXAMPLE 1.1

Draw the diagram for the binary tree with linear notation below.

(44 (22 () (98 () ())) (33 (99 () ()) ()))

We can see immediately that the root is 44. The next open bracket shows the start of the left subtree. So for the left subtree:

22 is the root;

the open bracket then close bracket show that it has an empty left subtree;

the right subtree consists of a single root 98—the following two pairs of brackets show its empty left and right subtrees.

The next open bracket shows the start of the right subtree. This has:

root 33;

left subtree consisting of the single node 99;

empty right subtree.

The tree with this linear notation is shown in Figure 1.13.

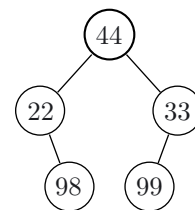


Figure 1.13

We could give a procedure for getting from the linear notation to the tree diagram, but shall not do so. We shall not ask you to do this for complicated trees. (You can see that the task will become increasingly complicated for larger trees.)

1.2 The class BinTree of X

We now turn to the question of how to construct and use a binary tree object in code. M263 makes available a library class **BinTree of X**. As with the classes **Vector** in Unit 6 and **Set** in Unit 7, we will be concerned with the specification of the class, not with how it is implemented. Like those classes, **BinTree of X** is a parameterised class, with data items taken from an unspecified class **X**. (In our examples, **X** will be **Int**, **Char** or **mString**.)

BinTree of X	Class specification
<i>States</i> The state of an object is a binary tree, which has a data item from X stored at each node. <i>State notation</i> The state of an object will usually be shown graphically, but we can use the linear notation introduced above. <i>Default</i> The empty tree. <i>Methods</i> <i>setTree</i>(<i>item</i> in X, <i>l</i> in BinTree, <i>r</i> in BinTree) state change <i>setRoot</i>(<i>item</i> in X) state change <i>setLeftTree</i>(<i>newLeft</i> in BinTree) state change <i>setRightTree</i>(<i>newRight</i> in BinTree) state change <i>getRoot</i>() return in X <i>getDepth</i>() return in Int <i>isEmpty</i>() return in Bool <i>leftTree</i>() return in BinTree <i>rightTree</i>() return in BinTree	For convenience, we have shortened BinTree of X to BinTree in the method signatures. As with all M263 classes, BinTree has access to <i>clone</i> via Base. These four methods change the state of a binary tree. These three methods return information about a tree. These two methods return new binary trees.

We now give specifications of the methods. These have been grouped in the class specification according to their nature. The first group of methods are used to change the state of the receiving tree object. The first of these enables one to set the state of the entire binary tree.

method *setTree*(*item* in **X**, *l* in **BinTree**, *r* in **BinTree**) state change
pre true.

post Changes the state of the receiver to a binary tree whose root is *item*, whose left subtree is *l* and whose right subtree is *r*.

The next three methods allow one to set the state of a tree by just changing a single feature: either the root, the left subtree or the right subtree.

method *setRoot*(*item* in **X**) state change
pre true.

post If the receiver is an empty tree, then its state is changed to a tree whose root is *item*, and whose left and right subtrees are empty. If the receiver is not empty, then its state is changed by changing the root element to *item*, with the left and right subtrees remaining unchanged.

method *setLeftTree*(*newLeft* in **BinTree**) state change
pre The receiver is not empty.

post Changes the state of the receiver by changing the left subtree to *newLeft*. The root and the right subtree are unchanged.

method *setRightTree*(*newRight* in **BinTree) state change****pre** The receiver is not empty.**post** Changes the state of the receiver by changing the right subtree to *newRight*. The root and the left subtree are unchanged.

After declaring a tree variable (**var *t* in BinTree of X**), to use these three methods to set *t* to a particular state one should apply *setRoot* first. This is the only method of the three that can be applied to an empty tree. After a call *t.setRoot(item)*, *t* will be non-empty, and *setLeft* and *setRight* can then be applied.

The other methods return information about a tree object. You will notice that the only stored item which we can readily access is that at the root.

method *getRoot*() return in X**pre** The receiver is not empty.**post** The returned value is the item stored at the root node in the receiver.**method *getDepth*() return in Int****pre** true.**post** The returned value is the depth of the receiver. (That is, 0 for an empty tree; the number of levels for a non-empty tree.)**method *isEmpty*() return in Bool****pre** true.**post** The returned value is **true** if the receiver is the empty tree and is **false** otherwise.**method *leftTree*() return in BinTree****pre** true.**post** If the receiver is not empty, then its left subtree is returned. If the receiver is an empty tree, then the returned value is an empty tree.**method *rightTree*() return in BinTree****pre** true.**post** If the receiver is not empty, then its right subtree is returned. If the receiver is an empty tree, then the returned value is an empty tree.

We can readily access the item stored at the root of a tree. By judicious use of the methods, we can access any item in a tree, but this may need a little thought. To access the item stored at a particular node in a tree, we will need to follow the unique path down the branches from the root to the desired node, going left or right as necessary. For example, look at the tree in Figure 1.14. To access the item 55 in that tree, we need to go: right, then left, and then extract the root. So if a variable *t* of type **BinTree of Int** is in the state shown in Figure 1.14, then the following expression will extract that integer 55.

```
t.rightTree().leftTree().getRoot()
```

This shows a form of coding that we can use with the class **BinTree**, but not with a class like **Spot**. Remember that *rightTree* and *leftTree* each returns a binary tree — they do not change the state of the receiver. So *t.rightTree()* does not change the state of the tree *t*; rather, it represents a value that is itself a binary tree. We can apply a method such as *leftTree* to this value. So we can “chain” methods, as in the expression above. That expression is valid in code. Of course, we would need to store the value that is returned in a suitable variable if we want it to be available later.

We can recognise an empty tree by the fact that its depth is 0, so we could manage without *isEmpty* — but use of this method can clarify code.

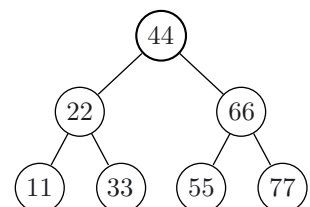


Figure 1.14

Activity 1.4

Suppose that t is a variable of type **BinTree of Int**, and is in the state shown in Figure 1.14. Give each of the following values. (For the tree values, show the trees graphically.)

- $t.getRoot()$
- $t.getDepth()$
- $t.leftTree()$
- $t.rightTree()$
- $t.leftTree().getRoot()$
- $t.leftTree().leftTree().getRoot()$
- $t.rightTree().leftTree().leftTree().isEmpty()$

Discussion

- The returned value is 44: the value at the root node.
- The returned value is 3: the depth of the tree.
- The returned value is shown in Figure 1.15: the left subtree of t .
- The returned value is shown in Figure 1.16: the right subtree of t .
- The returned value is 22: the root of the left subtree of t .
- The returned value is 11: the root of the left subtree of the left subtree of t .
- The returned value is **true**. The left subtree of the right subtree of t is a tree with the single node 55 (as in Figure 1.17), and the left subtree of that is an empty tree.

We can use the methods of **BinTree** to set the state of a tree to a particular value. To do this, it is useful to have a notation for an empty tree that we can use in code. This is available: the notation $ET()$ can be used in code. For example, for a variable t of type **BinTree of Int**, the code statement below will set the state of t to the tree shown in Figure 1.17 (with root 55, and empty left and right subtrees).

```
t.setTree(55,ET(),ET())
```

The notation $ET()$ suggests that ET is a function, and indeed it is, but it is rather different from the functions that we have previously discussed in this course. In Unit 1, we explained that to describe a function we need three pieces of information.

1. A signature of the function that includes the set from which the input will be drawn and the set from which the output is drawn.
2. A precondition, giving any restriction on the input.
3. A postcondition, giving the relationship between the output and the input.

The function $ET()$ differs from the functions that we have previously discussed in this course, in that it does not require any input. A function with no input inevitably will always return the same value. This is true for $ET()$. Whenever the function $ET()$ is executed, it does exactly the same

When answering questions, always show trees graphically unless explicitly asked to use the linear notation.

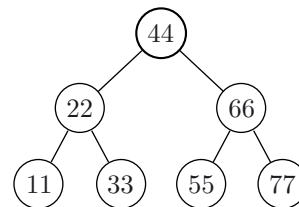


Figure 1.14

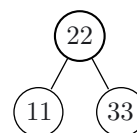


Figure 1.15

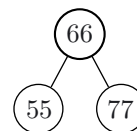


Figure 1.16



Figure 1.17

thing; that is, it creates an empty tree. The value returned as $ET()$ is the empty tree, a value of type **BinTree**.

A description of the function ET in standard form is given below. As there are no inputs required by this function, the precondition must be **true**.

function $ET()$ **return in** **BinTree**

pre **true**.

post The returned value is the empty binary tree.

The signature of the mathematical function ET can be written as $ET : \rightarrow \mathbf{BinTree}$, showing that no input is required.

We saw above how to write code to set up a tree of depth 1. To construct larger trees the process rapidly becomes rather laborious. For example, the code below will set the state of the variable t to the tree shown in Figure 1.15. To do this, we first set up the left and right subtrees (which are of depth 1), and then use these to set the tree itself.

```
var t in BinTree of Int
var leftT in BinTree of Int
var rightT in BinTree of Int
leftT.setTree(11, ET(), ET())    // set the left subtree
rightT.setTree(33, ET(), ET())  // set the right subtree
t.setTree(22, leftT, rightT)    // finally, set the tree
```

Activity 1.5

Write a code fragment that sets the states of variables $aTree$, $bTree$ and $cTree$ to the trees shown in Figure 1.18 (a), (b) and (c) respectively.

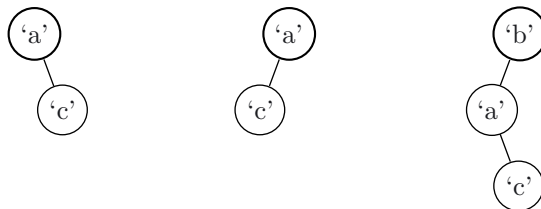


Figure 1.18 (a)

(b)

(c)

Remember that the trees in Figure 1.18 (a) and (b) are not the same!

Discussion We start by creating a variable $temp$ and setting its state to a tree with just one node, with 'c' stored at its root. We can then build the three required trees in turn.

```
var temp in BinTree of Char
temp.setTree('c', ET(), ET())
aTree.setTree('a', ET(), temp)
bTree.setTree('a', temp, ET())
cTree.setTree('b', aTree, ET())
```

As we can see, it requires some work to set up even quite a small tree this way. In the next section we will consider some processes whereby we can set up larger trees in a rather more simple fashion.

To draw the tree diagram representing the state of a tree variable produced by a given code fragment is more straightforward.

Activity 1.6

Determine the state of the variables after the code fragment below has been executed.

```
var t in BinTree of Int
var u in BinTree of Int
var v in BinTree of Int
var w in BinTree of Int
v.setTree(55, ET(), ET())    //(1)
t.setTree(11, v, v)          //(2)
w.setTree(66, t, t)          //(3)
u.setTree(11, v, w)          //(4)
```

Discussion With tree variables, it is often inconvenient to lay out a trace of given code in a table (as we have done for other types in earlier units).

After the four **var** statements all the trees are empty (in the default state).

Statement (1) sets *v* to a tree with the single node 55 (see Figure 1.19).

Statement (2) then sets *t* to the tree shown in Figure 1.20(a).

Statement (3) sets *w* to the tree in Figure 1.20(b).

Finally, statement (4) sets *u* to the tree shown in Figure 1.20(c). Notice how that tree is made up:

the root is 11;

the left subtree consists of a tree with a single node 55 (the tree *v*);

the right subtree is the tree *w*, as shown in Figure 1.20(b).

55

Figure 1.19

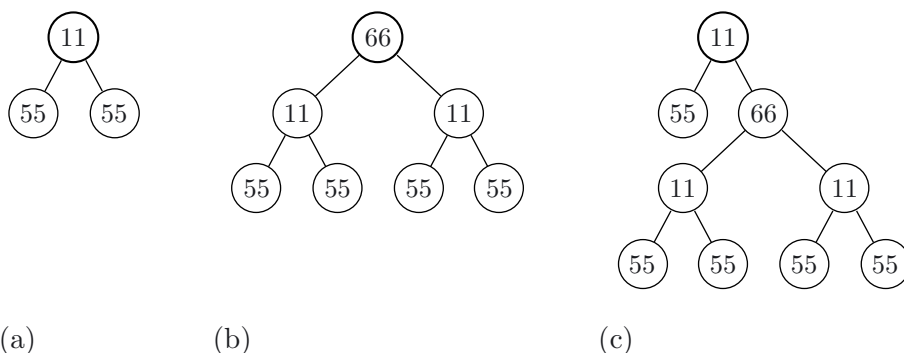


Figure 1.20 (a) *t* after (2). (b) *w* after (3). (c) *u* after (4).

As well as the *setTree* method, **BinTree** has three other methods that change the state of a tree variable by setting one of its three features (the root, the left subtree or the right subtree) to a value supplied as a parameter, leaving the other two features unchanged. For instance, the statement *t.setRoot*(99) changes the state of *t* by changing the root value to 99, leaving the left and right subtrees unchanged.

Activity 1.7

Suppose that t is a variable of type **BinTree of Int** whose state is as shown in Figure 1.21. Give diagrams showing the state of t and u after the execution of each statement in the code fragment below.

```
var u in BinTree           //(1)
t.setRoot(t.leftTree().getRoot()) //(2)
u <-- t.rightTree()        //(3)
t.setLeftTree(u)           //(4)
```

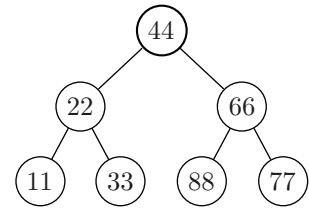


Figure 1.21

Discussion After (1), u is an empty tree, with the state of t unchanged.

Statement (2) changes the integer at the root of t . The new value is the integer found at the root of the left subtree of t in Figure 1.21, which is 22. So the state of t after (2) is as shown in Figure 1.22(a).

Statement (3) does not change the state of t . The state of u is changed to become the right subtree of the tree in Figure 1.22(a). This is shown in Figure 1.22(b).

Statement (4) then changes the state of t , by replacing its left subtree by the current value of u . This leaves t in the state shown in Figure 1.22(c).

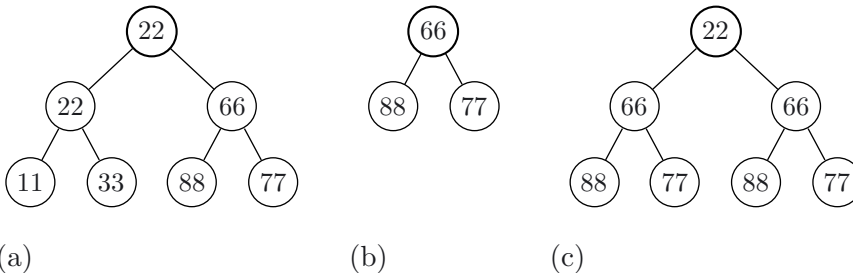


Figure 1.22 (a) t after (2). (b) u after (3). (c) t after (4).

We can use the methods of **BinTree** to implement general processes, which we can specify as functions. For example, consider the function *SWAPLEFT*, specified below.

function *SWAPLEFT*(t in **BinTree of X**) **return** in **BinTree of X**

pre t is not empty and has a non-empty left subtree.

post The returned value is a binary tree that is formed from the input tree t by exchanging the root of t with the root of the left subtree of t .

We can implement this using the following strategy.

First declare a tree variable and set it to a copy of t .

Then store the value at the root in a suitable variable.

Then set the root to the value at the root of the left subtree.

Finally, we need to reset the value at the root of the left subtree. To do this:

- set a new binary tree variable to the value of the left subtree;
- reset its root value to the value that has been stored;
- then use this to reset the left subtree.

Activity 1.8

Give code to implement the function *SWAPLEFT* specified on page 17.

Discussion The implementation uses the strategy outlined above.

```
function SWAPLEFT(t)
{
  var outTree in BinTree of X
  var theRoot in X
  var newLeft in BinTree of X
  outTree <-- t.clone()
  theRoot <-- t.getRoot() // store the root
  outTree.setRoot(t.leftTree().getRoot())
  /* change the root to the root of the left subtree */
  newLeft <-- t.leftTree()
  /* newLeft is now the left subtree of the input tree */
  newLeft.setRoot(theRoot)
  /* newLeft now has the correct root */
  outTree.setLeftTree(newLeft)
  return outTree
}
```

Objectives for Section 1

After studying this section you should be able to do the following.

- ☐ Recognise and use terminology relating to binary tree diagrams: node, left and right branches, subtree, parent node, child node, leaf node, level and depth of a tree.
- ☐ Interpret the linear notation used to show a binary tree.
- ☐ Work with the class **BinTree**, and in particular give:
 - a diagram to show the state of an object of this class after given code statements have been executed
 - code statements to set up a variable of this type whose state is a given small tree
 - code to extract data from or to manipulate an object of the class.
- ☐ Understand that a binary tree can be described in terms of its root node and its left and right subtrees.

Exercises on Section 1

Exercise 1.1

Suppose that the variable t has the state shown in Figure 1.23.

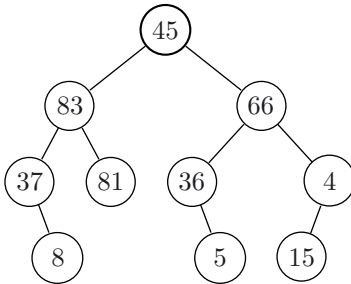


Figure 1.23

- (a) Write down the value that will be returned by each of (i)–(iv).
- (i) $t.getDepth()$
 - (ii) $t.rightTree().leftTree().getRoot()$
 - (iii) $t.leftTree().rightTree().getRoot()$
 - (iv) $t.rightTree().rightTree().rightTree().isEmpty()$
- (b) Draw a diagram showing the state of t after the following fragment of code has been executed.

```

var temp in BinTree of X
temp<--t.leftTree()
t.setLeftTree(t.rightTree())
t.setRightTree(temp)
  
```

Exercise 1.2

Give an implementation of the function *DELETERIGHT*, specified below.

function *DELETERIGHT*(t in **BinTree** of **X**) **return** in **BinTree** of **X**
pre t is not empty.
post The returned value is a binary tree that has the same root and left subtree as t , but has an empty right subtree.

Exercise 1.3

Give an implementation of the function *LEFTLEAF* specified below.

function *LEFTLEAF*(t in **BinTree** of **X**) **return** in **X**
pre t is not empty.
post The returned value is the leaf node of t that is furthest left.

For example, for the tree t in Figure 1.23, *LEFTLEAF*(t) would be 8.

To implement *LEFTLEAF*, use the following strategy.

Set a local variable $temp$ of type **BinTree** to a copy of t .

So long as either the left or right subtree of $temp$ is not empty, do the following:

if the left subtree of $temp$ is not empty, replace $temp$ by its left subtree;
 otherwise, replace $temp$ by its right subtree.

Finally, return the root of $temp$.

2 Binary trees and vectors

Given a tree, one can write the items stored at its nodes in a sequence. To do this, one needs to visit each node in the tree exactly once, and copy into the sequence the item at that node as it is visited. However, there is no single obvious way of doing this. Any systematic way of visiting each node in a tree once is called a **tree traversal**. There are several fairly natural ways of traversing a tree, and we shall look at two of these ways in Subsection 2.2.

We shall also look in this section at two ways of moving from a sequence to a tree. In doing this, we will want to place each item in the sequence at a node in a binary tree. Again, there are several ways in which this can be done. We start (in Subsection 2.1) by looking at an approach to this that is quite easy to describe and execute. However, a tree stores the data items in a structured way, and one is likely to want to place the items in the sequence into a tree in a useful way. We will look in Subsection 2.3 at a way of moving data from a sequence to a tree that is more subtle, but that places the data in the tree in a way that can be particularly useful.

In Unit 6, Section 4, we mentioned briefly a way of searching in a sorted sequence (the “halving strategy”) that is more efficient than examining the items one-by-one, starting at the front. The approach that we shall discuss in Subsection 2.3 will store the items from a sequence in a tree in a way that is convenient for this efficient approach to searching. In such a search, the number of steps required depends on the depth of a tree. To exploit this efficiency, we want to store the items in a tree in a balanced way.

2.1 Complete trees

Suppose we have a sequence, and store the items in a tree in the following way. Put the first item at the root; put the second item as the right child of the root, put the third item as the right child of that, and so on, so that each item is stored as the right child of the previous item. Then the sequence [11, 22, 33, 44] would be stored in the tree shown in Figure 2.1. Notice that the depth of that tree is the same as the length of the sequence [11, 22, 33, 44]. Such an approach to storing items in a tree does not exploit the storage capability of a tree in a useful way.

One test that a tree is not badly “lop-sided” is that it is balanced, which is defined as follows.

A tree is **balanced** if, below each node, the left and right branches lead to subtrees whose depths do not differ by more than 1.

To see whether a given tree is balanced, we need to consider each node in turn, and to find the depth of the two subtrees below it.

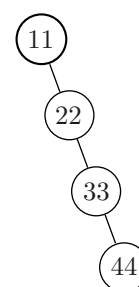


Figure 2.1

Some authors use slightly different definitions of the term “balanced”.

Activity 2.1

Which of the trees in Figure 2.2 are balanced?

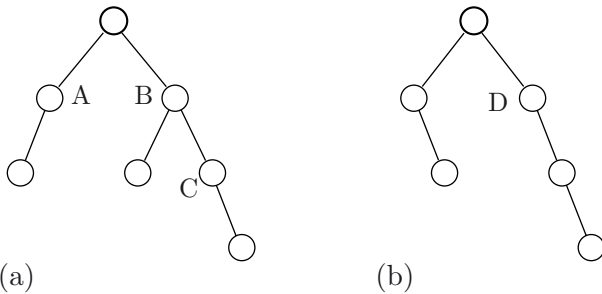


Figure 2.2

Discussion (a) is balanced but (b) is not. For the tree in (a), the depths of the subtrees below each node are as follows. The root: left 2, right 3; node A: left 1, right 0; node B: left 1, right 2; node C: left 0, right 1. In (b), the test fails at the node D, where the left subtree has depth 0 and the right subtree has depth 2.

We do not need to know what data is stored at the nodes to decide whether a tree is balanced.

Leaf nodes do not need to be considered, since below a leaf node both subtrees will have depth 0.

A balanced tree avoids serious inefficiency in storage, but it does not ensure that a tree is as full as possible for a given depth. For example, in the tree in Figure 2.2(a), a right child could be added below the node A. We will say that a level in a tree is completely filled if there are as many nodes at that level as there could possibly be. More formally, we can say that in a non-empty tree, level n is **completely filled** if either:

1. $n = 1$; or
2. level $n - 1$ is completely filled, and every node at that level has two children.

Suppose that a tree of depth n has all its levels completely filled. We can count the number of nodes in such a tree as follows. At level 1 there is just one node, the root. The root will have two children, so there are two nodes at level 2. Each of these has two children, so there are four nodes at level 3, and so on. At each level there are twice as many nodes as there are at the level above. So there are: $1 = 2^0$ nodes at level 1; $2 = 2^1$ nodes at level 2; $4 = 2^2$ nodes at level 3; $8 = 2^3$ nodes at level 4; and so on. At level i (for $i \leq n$) there will be 2^{i-1} nodes.

The total number of nodes in the tree would be:

$$1 + 2 + 2^2 + 2^3 + \dots + 2^{n-1}.$$

This sum can be shown to be equivalent to the formula $2^n - 1$.

For example, you might like to verify that (with $n = 5$):

$$1 + 2 + 4 + 8 + 16 = 32 - 1.$$

Now if we are transferring data from a sequence into a tree, we will only be able to fill every level in the tree if the sequence contains exactly $2^n - 1$ items (for some integer n), which is unlikely to be the case. However, we can always ensure that we transfer data to the tree so that we fill up each level before we start on a new one. If we fill a tree of depth n in such a way, then it will contain between 2^{n-1} and $2^n - 1$ items. For example, 2^{20} is over a million, so we can store a million items in a tree of depth 20. In such a tree, the root node is no more than 19 steps away from any other node. To

In a non-empty tree there is a root, and there is no room for anything else at level 1

See Unit 14.

exploit this structure to its fullest potential we now consider ways of placing data items into a tree which ensure that the tree is densely filled.

We describe first a very natural way of transferring data stored in a sequence to such a tree. This places the data in a **complete** tree. A complete tree is one in which all but one of its levels is completely filled. We will also require that the bottom level in a complete tree is being filled in a systematic way, working from the left. This process for placing the items in a given sequence into a complete tree is illustrated for the sequence [11, 22, 33, 44, 55, 66, 77] in Figure 2.3, and laid out below as a procedure.

We will also say that an empty tree is complete.

Strategy 2.1

To form a complete tree from a sequence s

1. Work from the left of the sequence s .
2. Put the first item at the root node of the tree.
3. Add the second item as the left child of the root and the third item as the right child of the root.
4. Move to level 3 in the tree, and add the next four items at that level, again working from the left.
5. Move to level 4 in the tree, and fill that starting at the left.
6. Continue until all the items in the sequence have been entered in the tree.

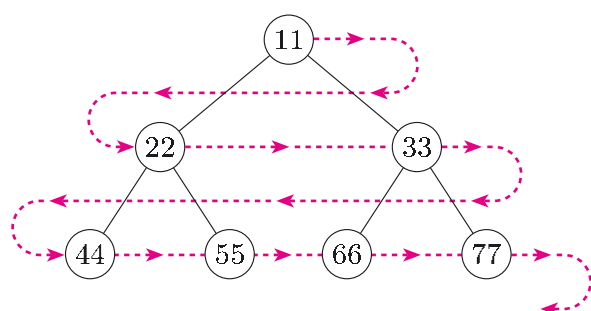


Figure 2.3

In Figure 2.3, levels 1 to 3 have been completely filled. The items from the sequence [11, 22, 33, 44, 55, 66, 77] were added to the tree as indicated by the dotted arrow. We start at the root and work down one level at a time. Each level is filled from left to right. If the sequence contained another element, say 88, this would be added to the tree as the left child of 44, starting another level in the tree.

Activity 2.2

Use Strategy 2.1 to form a complete tree from the sequence [2, 44, 63, 12, 8].

Discussion The resulting tree is shown in Figure 2.4. To obtain that, follow the pattern in Figure 2.3. First place the value 2 at the root node. Then fill the second level completely with the values 44 and 63 from the sequence. Then move to the third level and start to fill it from the left, so that the values 12 and 8 are added as shown.

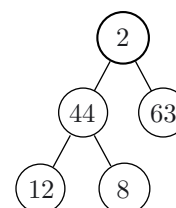


Figure 2.4

Activity 2.3

Which of the binary trees in Figure 2.5 are balanced? Which are complete?

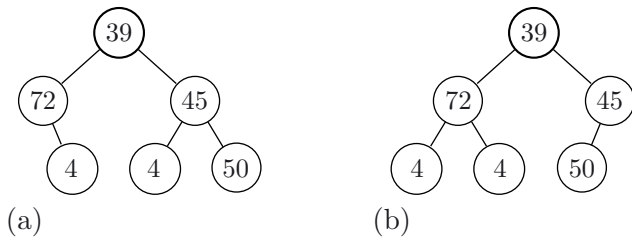


Figure 2.5

Discussion Both trees are balanced. The tree in (b) is complete, but that in (a) is not. In (a), the nodes at level 3 are not filled from the left. The node storing 72 could have a left child. In (b), level 2 is fully filled, and level 3 is filled from the left.

Any complete tree must be balanced.

We now give a specification of a function that returns a complete tree, constructed from a sequence using Strategy 2.1. In function specifications in this section, we shall treat sequences as vectors.

function *COMPLETE_TREE*(*v* in **Vector of X**) **return** in **BinTree of X**
pre true.

post The returned tree is a complete tree. The nodes are filled with the elements of the vector *v* using Strategy 2.1 (as illustrated in Figure 2.3).

So, for example, if *v* is the vector [39, 72, 45, 4, 4, 50], *COMPLETE_TREE*(*v*) will return the tree shown in Figure 2.5(b).

We shall not give an implementation of the function *COMPLETE_TREE* (nor of any other functions that we introduce in this section).

Most of our discussion is only concerned with a sequence as a form of data, and so applies equally well to other sequence-based types.

The function *COMPLETE_TREE* is available in the WorkPad.

Activity 2.4

Draw the tree diagrams of the trees returned by *COMPLETE_TREE*(*v*), where *v* is each of:

- (a) [11, 22, 33, 44, 55, 66, 77, 88];
 (b) [11, 22, 33, 44, 55, 66, 77, 88, 99, 110].

Discussion The trees are shown in Figure 2.6. In each case, we work through the vector from the front, first placing 11 at the node of the tree. The first three levels of the tree are completely filled, with 22 and 33 placed at the nodes in level 2, while 44, 55, 66 and 77 are placed at the nodes in level 3. Finally, the level 4 is filled from the left, one node at a time.

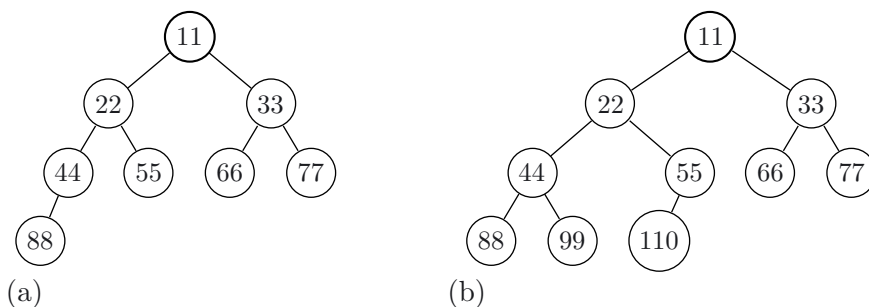


Figure 2.6

2.2 Tree traversals

A tree traversal is a systematic way of visiting each node in a tree. There are a number of purposes for which one might traverse a tree, for example to search to see whether some item is present in the tree. We shall look here at tree traversals as a way of transferring the items stored at the nodes of a tree into a sequence — so reversing the process encapsulated in *COMPLETE_TREE* in the previous subsection (which transferred data from a sequence to a tree). There are a number of forms of tree traversal. Here we shall consider two, called level order and in-order.

We consider **level order** traversal first. In this, the nodes in a tree are visited (and added to a sequence) starting at level 1, then visiting each level in turn down to the deepest level of the tree. Each level is traversed from left to right. This process is exemplified in Figure 2.7, and set out below as Strategy 2.2.

This traversal is sometimes referred to as **breadth first** traversal, indicating that we move breadth-wise across each level in the tree.

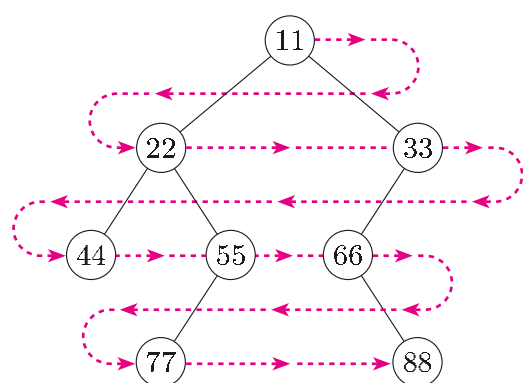


Figure 2.7

Strategy 2.2

Level order tree traversal

Given a tree, we form a sequence containing the items at its nodes as follows. (The sequence is written down going from left to right.)

First, the root.

Then the nodes at level 2, working from left to right.

Then the nodes at level 3, working from left to right.

This process is continued, going down one level at a time, until all the nodes in the tree have been visited.

We specify below a function corresponding to level order tree traversal.

function *LEVEL_ORDER*(*t* in BinTree of X) **return** in Vector of X
pre true.

post The returned value is a vector containing the elements stored at the nodes of *t*, written in the order determined by Strategy 2.2.

The function *LEVEL_ORDER* is available in the WorkPad.

Level order traversal reverses Strategy 2.1, for forming a complete tree from a sequence. However, level order traversal can be applied to any tree, not just complete trees. If we start with a sequence *v*, apply *COMPLETE_TREE* to that, and then apply *LEVEL_ORDER* to the tree we obtain, then we will end up with the same sequence *v*. But if we start with a tree *t*, apply *LEVEL_ORDER* to that, and apply *COMPLETE_TREE* to the sequence we obtain, then we will not usually end up with the same tree. We will end up with a complete tree, which will be the same as the original tree only if

that was itself complete. For example, the tree t shown in Figure 2.7 is not complete. For that tree t , $LEVEL_ORDER(t)$ is $[11, 22, 33, 44, 55, 66, 77, 88]$. Applying $COMPLETE_TREE$ to this sequence will give the tree in Figure 2.6(a), not the tree shown in Figure 2.7.

Activity 2.5

Suppose that $aTree$, $aVect$ and $aBool$ are variables of type **BinTree of Int**, **Vector of Int** and **Bool** respectively, and that $aTree$ is in the state shown in Figure 2.8. Write down the states of $aVect$ and $aBool$ after the code fragment below has been executed.

```
aVect <-- LEVEL_ORDER(aTree)
aBool <-- aVect.contains(17)
```

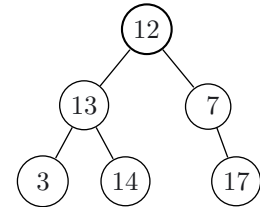


Figure 2.8

Discussion The states are:
 for $aVect$, $[12, 13, 7, 3, 14, 17]$;
 for $aBool$, **true**.

The level order traversal reads: first the root, 12, then the two nodes at level two, reading from the left (13, 7), then the nodes at level three (3, 14, 17), reading from the left.

The call of *contains* checks whether the parameter, 17, appears in the vector $[12, 13, 7, 3, 14, 17]$. It does, so the returned value is **true**.

Activity 2.6

Suppose that u and v are of type **Vector of Int**, t is of type **BinTree of Int**, and the code fragment below is executed. Give the final states of u and t . Are the final states of the two vectors u and v the same?

```
v.setVector(11,22,33,44,55,66)
t <-- COMPLETE_TREE(v)
u <-- LEVEL_ORDER(t)
```

Discussion Applying the function $COMPLETE_TREE$ to the vector $[11, 22, 33, 44, 55, 66]$ produces the tree shown in Figure 2.9, so this is the final state of t . Applying $LEVEL_ORDER$ to this tree produces the vector $[11, 22, 33, 44, 55, 66]$, so this is the final state of u . The final states of u and v are the same.

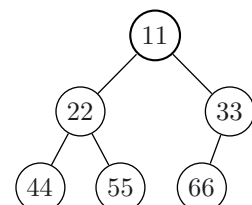


Figure 2.9

We now turn to a different systematic way of visiting each node in a tree, called **in-order** traversal. This is more difficult to describe than level order traversal, and is best understood by reference to a diagram. In Figure 2.10(a), the numbers below each node indicate the order in which the nodes are visited (they are *not* the elements stored at the nodes).

An in-order traversal of the tree in Figure 2.10(a) produces the sequence ['d', 'b', 'e', 'a', 'c']. This sequence is obtained as follows.

Follow the arrow that is shown in Figure 2.10(a) around the outside of the tree. When you pass underneath a node, add it to the sequence. We pass first beneath the node containing 'd', and numbered 1; second beneath the node containing 'b', and numbered 2, and so on.

You may see a diagram such as that in Figure 2.10(a) referred to as a **tree walk** diagram.

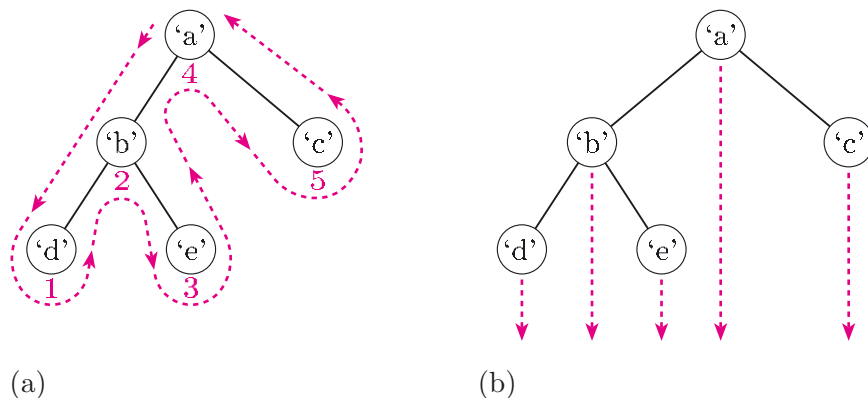


Figure 2.10

An in-order traversal produces a sequence in a way that is consistent with the “left-to-right” structure of a binary tree. Everything in the left subtree appears before the root and the root appears before everything in the right subtree. What is more, this comment is true at every node in the tree, not just at the root. Suppose that we draw the diagram of a tree very carefully, as illustrated in Figure 2.10(b). There, the nodes are spread out so as to ensure that we can draw a line vertically down from each node, without these lines crossing the tree. Then we will obtain the sequence produced by an in-order traversal by “squashing” the diagram vertically. This process is referred to as **pancaking**.

Informally, the procedure for writing down the sequence produced by in-order traversal is given in Strategy 2.3.

Strategy 2.3

In-order tree traversal

Draw a line around the outside of the tree. Start to the left of the root, and go around the outside of the tree, to end up to the right of the root. Stay as close to the tree as possible, but do not cross the tree. (Think of the tree — its branches and nodes — as a solid barrier.) The order of the nodes is the order in which this line passes underneath them. If you are unsure as to when you go “underneath” a node, remember that a node “to the left” always comes first.

For example, in the tree in Figure 2.8 the node storing 7 is “to the left” of the node storing 17. The in-order traversal of that tree gives the sequence [3, 13, 14, 12, 7, 17].

Alternatively, draw your tree diagram in a way that allows you to use pancaking. If you draw the tree correctly, then you can draw a line vertically down from each node, and these lines will not cross the tree.

See Figure 2.10(b).

In-order traversal is rather awkward to formalise, either as a specific procedure or as code. The essence of the idea is this.

Form a sequence as follows. Write down the in-order traversal of the left subtree, then write down the root, then write down the in-order traversal of the right subtree. (The in-order traversal of an empty tree is the empty sequence.)

This characterisation is recursive! That is, it describes the in-order traversal by referring to itself. This self-reference works because of two things. The in-order traversal of a tree is described in terms of the in-order traversal of other, smaller, trees. And by giving the in-order traversal of an empty tree separately, we show how this process of self-reference will stop. We shall discuss recursion in Unit 13, and do not want to pursue recursive ideas to any great extent here. However, it is difficult to discuss trees without mentioning recursion, since the structure of a tree is intrinsically recursive. As we noted in Section 1, a tree is either empty, or it is: a tree (the left subtree), and an item (the root), and a second tree (the right subtree). The recursive nature of the structure means that many ideas associated with trees are best thought of recursively. It also means that many processes associated with trees are most conveniently implemented using recursion.

A function *IN_ORDER* is specified below.

function *IN_ORDER*(*t* in **BinTree** of **X**) **return** in **Vector** of **X**

pre **true**.

post The returned value is a vector containing the elements stored at the nodes of *t*, given in the order determined by an in-order traversal.

That is one reason why we have not given implementations here of the various functions on trees that have been discussed.

The function *IN_ORDER* is available in the WorkPad.

Activity 2.7

Give *IN_ORDER*(*t*), where *t* is each of the trees given in:

- (a) Figure 2.9;
- (b) Figure 2.6(a).

Discussion The sequences obtained are:

- (a) [44, 22, 55, 11, 66, 33];
- (b) [88, 44, 22, 55, 11, 66, 33, 77].

2.3 Growing trees by division

The functions *COMPLETE_TREE* and *LEVEL_ORDER* are closely related. Starting with a sequence *s*, if we first perform *COMPLETE_TREE*, and then perform *LEVEL_ORDER* on the resulting tree, we will obtain a sequence identical to the starting sequence *s*.

This is illustrated in Activity 2.6.

We now look at a way of forming a tree from a sequence that pairs in a similar way with *IN_ORDER*. We shall call the function corresponding to this process *GROW_TREE*. This function puts the items from the sequence into a tree in a way that reflects within the tree structure the order of the items in the sequence. For example, in the sequence [11, 22, 33, 44, 55, 66, 77], 33 appears to the left of 66. We will grow the tree in a way that ensures that 33 is to the left of 66 in the tree.

We will also grow the tree in a way that produces a balanced tree. To do this we will take as the root node an item in the sequence as close to the centre as possible. We shall start by describing how the procedure to grow a tree works for a particular example: the sequence $[11, 22, 33, 44, 55, 66, 77]$. This is illustrated in Figure 2.11.

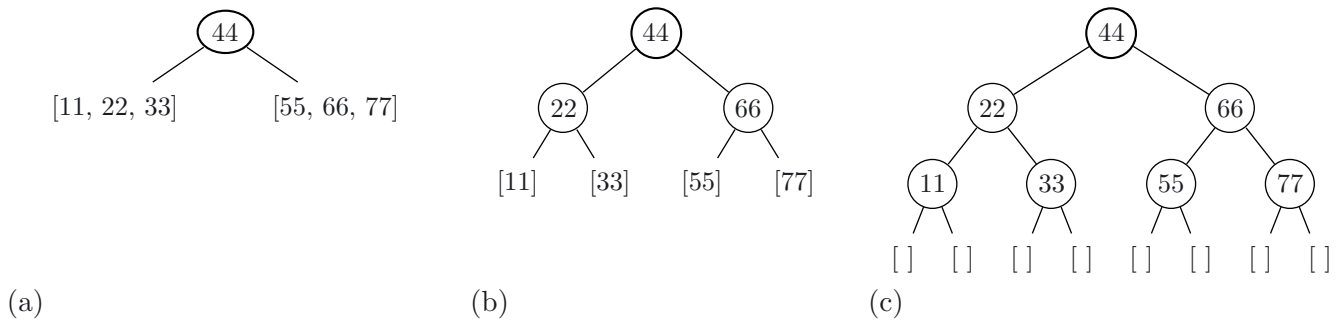


Figure 2.11

The first step is to choose the element nearest to the centre of the sequence. In this example, that element is 44. Now we can see the sequence $[11, 22, 33, 44, 55, 66, 77]$ in the following way: a sequence $[11, 22, 33]$, then the element 44, then a sequence $[55, 66, 77]$. To form the tree, we first place the element 44 at the root. The left subtree of the tree will be formed from the sequence $[11, 22, 33]$, and the right subtree of the tree will be formed from the sequence $[55, 66, 77]$, as indicated in Figure 2.11(a).

The next step is to treat each of the vectors $[11, 22, 33]$ and $[55, 66, 77]$ in a similar way. The element at the centre of $[11, 22, 33]$ is 22, and this sequence divides as follows: the sequence $[11]$; the item 22, then the sequence $[33]$. The element at the centre of $[55, 66, 77]$ is 66, and this sequence divides as: the sequence $[55]$, the item 66, then the sequence $[77]$. The second step is shown in Figure 2.11(b).

The item at the centre of the sequence $[11]$ is obviously 11, and we can divide this sequence as: $[], 11, []$. The sequences $[33]$, $[55]$ and $[77]$ divide up in a similar way (Figure 2.11(c)).

The process stops when we are left with only empty sequences to deal with. An empty sequence corresponds to an empty tree. Since we do not show empty subtrees in our tree diagrams, we remove the empty sequences to show the tree we have grown from $[11, 22, 33, 44, 55, 66, 77]$. This is given in Figure 2.12.

In this example, all the sequences split conveniently. Each had an odd number of members, and so had a central element. If we need to split a sequence with an even number of elements, then there will be two candidates for the “middle” element. In such a case, we will choose the element on the right of the two candidates.

For instance, in $[11, 22, 33, 44]$ we would choose 33 as the “centre” element. This splits the sequence into $[11, 22]$, 33 and $[44]$. In $[11, 22]$ we would choose 22 as the centre, which splits $[11, 22]$ into $[11]$, 22 and $[], 22, []$. The first step in the process of growing a tree from $[11, 22, 33, 44]$ is illustrated in Figure 2.13(a), and the tree we obtain is shown in Figure 2.13(b).

Below, we set out this strategy for growing a tree from a sequence.

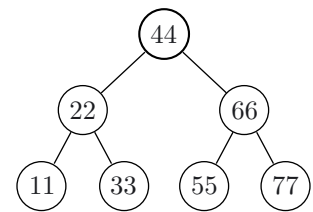


Figure 2.12

Remember that $[]$ denotes an empty sequence.

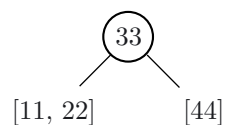


Figure 2.13 (a)

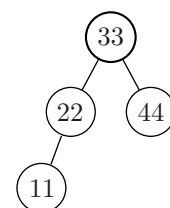


Figure 2.13 (b)

Strategy 2.4**Growing a tree from a sequence**

1. If the sequence is empty then the empty tree is returned.
2. If the sequence is not empty, first find the centre of the sequence.
(If the length of the sequence is even, then take the right-hand of the two candidates.)
3. Set this central element at the root of the returned tree.
4. The central element divides the sequence into a left sub-sequence and a right sub-sequence (one or both of which may be empty).
5. Apply the same procedure to the left sub-sequence to produce the left subtree of the returned tree.
6. Apply the same procedure to the right sub-sequence to produce the right subtree of the returned tree.

The process stops when there are only empty vectors to be dealt with.

We now specify a function corresponding to this process for growing a tree from a sequence.

function *GROW_TREE*(*v* in **Vector of X**) **return** in **BinTree of X**
pre true.

post The returned tree is a tree constructed according to Strategy 2.4.

For example, *GROW_TREE*(*s*) is the tree shown in Figure 2.12 when *s* is [11, 22, 33, 44, 55, 66, 77], and is the tree shown in Figure 2.13(b) when *s* is [11, 22, 33, 44]. Any tree obtained from Strategy 2.4 is balanced. Indeed, if the tree has depth *n*, then all the levels up to *n* – 1 will be completely filled. This much is like a complete tree, but the pattern of nodes that may be filled at the bottom level is different (and not easy to describe).

The function *GROW_TREE* is available in the WorkPad.

Activity 2.8

Draw the trees returned by *GROW_TREE*(*s*) if *s* is each of the following sequences.

- (a) [44, 22, 77, 55, 11, 66, 88, 33]
- (b) [11, 22, 33, 44, 55, 66, 77, 88, 99, 100]

Discussion

- (a) The “centre” of the sequence [44, 22, 77, 55, 11, 66, 88, 33] is the integer 11. This divides the sequence as follows: the sequence [44, 22, 77, 55], the integer 11, the sequence [66, 88, 33]. So the tree is formed as follows:
 - the root is 11;
 - the left subtree is formed from the sequence [44, 22, 77, 55];
 - the right subtree is formed from [66, 88, 33].

The sequence [44, 22, 77, 55] has “centre” 77, and splits as:
 [44, 22]; 77; [55].

The tree grown from [44, 22] has 22 as its root, left subtree just 44, and right subtree empty.

The tree grown from [66, 88, 33] has root 88, left subtree just 66 and right subtree just 33.

When we say that a tree is “just 44”, we mean it has root 44 and empty left and right subtrees.

So we obtain the tree shown in Figure 2.14(a).

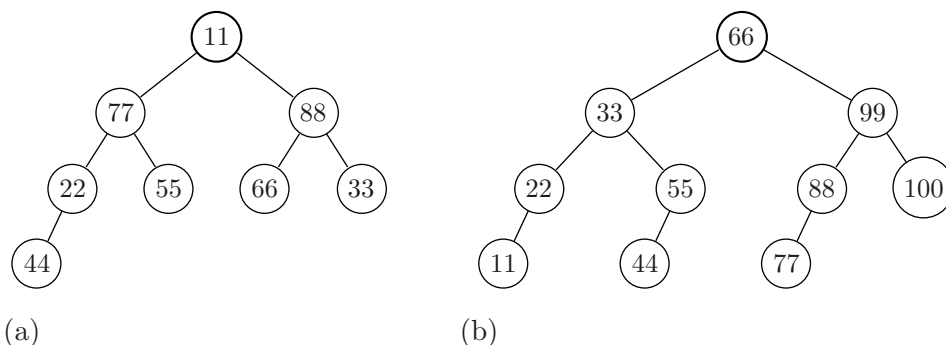


Figure 2.14

(b) The required tree is shown in Figure 2.14(b).

The “centre” of $[11, 22, 33, 44, 55, 66, 77, 88, 99, 100]$ is 66 (the root in Figure 2.14(b)). This sequence divides as: $[11, 22, 33, 44, 55]$; 66, $[77, 88, 99, 100]$.

The “centre” of $[11, 22, 33, 44, 55]$ is 33. This divides as: $[11, 22]$; 33; $[44, 55]$.

The tree grown from $[11, 22, 33, 44, 55]$ gives the left subtree in Figure 2.14(b).

The “centre” of $[77, 88, 99, 100]$ is 99. This divides as: $[77, 88]$; 99; $[100]$.

The tree grown from $[77, 88, 99, 100]$ is the right subtree in Figure 2.14(b).

Notice that all the trees obtained from *GROW_TREE* in our examples have all but the bottom level completely filled, as remarked above. You may also notice that the bottom level is filled in a way that is “biased to the left”. This is a result of the way we chose to pick the “centre” element when the length of the sequence is even.

The sequence in Activity 2.8(b) is sorted, as were several of our earlier examples. In such a case, the way we have grown the tree from the sequence ensures that all the numbers in the left subtree are smaller than the root, which in turn is smaller than all the numbers in the right subtree. Because of the recursive way we have grown these trees, every subtree also has this same property.

We introduced the idea of a sorted sequence in Unit 6. In a sorted sequence, the elements are arranged in increasing order from left to right.

Activity 2.9

For each of the trees t given in Figure 2.14(a) and (b), what is *IN_ORDER*(t)?

Discussion We obtain the sequences corresponding to in-order traversal by “pancaking” the tree in each case, and get:

(a) $[44, 22, 77, 55, 11, 66, 88, 33]$;

(b) $[11, 22, 33, 44, 55, 66, 77, 88, 99, 100]$.

These are the sequences from which the trees were grown in Activity 2.8. That should be no surprise. We designed this process of growing trees to correspond with the process of in-order traversal.

The functions *GROW_TREE* and *IN_ORDER* pair up in the same sort of way as do *COMPLETE_TREE* and *LEVEL_ORDER*. Starting with a sequence v , if we first perform *GROW_TREE*, then perform *IN_ORDER* on the resulting tree, we return to the original sequence, v .

However, if we start with a tree t , apply *IN_ORDER* to the tree, then apply *GROW_TREE* to that vector, we will not usually end up with the same tree. For example, if we start with the tree shown in Figure 2.15, and apply *IN_ORDER*, then we obtain the vector $[44, 22, 77, 55, 11, 66, 88, 33]$. Applying *GROW_TREE* to that vector gives the tree in Figure 2.14(a), and this is not the same tree. We can apply *IN_ORDER* to any tree, whether or not it is balanced, but the tree returned by *GROW_TREE* will always be balanced.

If we store data from a sequence in a tree using *GROW_TREE*, then we can recover the original sequence using *IN_ORDER*. All the information stored in the sequence has been preserved. However, if we start with a tree, then transfer its data to a sequence using *IN_ORDER*, and then move back to a tree using *GROW_TREE*, we do not recover the original tree. The same happens with *COMPLETE_TREE* and *LEVEL_ORDER*. This suggests that the tree has more intrinsic structure than a sequence, and if we move data from a tree to a sequence, it may not be possible to recover all the information originally stored in the tree. Indeed, to store all the information in a tree within a sequence, we need to use a very carefully structured sequence, with appropriate separators, as is done in the linear notation for a tree introduced in Section 1.

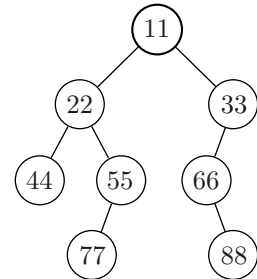


Figure 2.15

Objectives for Section 2

After studying this section you should be able to do the following.

- ☐ Recognise and use the terminology: balanced, completely filled, complete.
- ☐ Determine whether a given tree is balanced or complete.
- ☐ Determine the tree obtained when either of the functions *COMPLETE_TREE* or *GROW_TREE* is applied to a given vector.
- ☐ Determine the vector obtained when either of the functions *LEVEL_ORDER* or *IN_ORDER* is applied to a given tree.

Exercises on Section 2

Exercise 2.1

Determine the state of the tree t after the code fragment below has been executed. Is the tree you obtain balanced? Is it complete?

```

var v in Vector of Int
var t in BinTree of Int
v.setVector(11,22,33,44)
t.setTree(99, COMPLETE_TREE(v), GROW_TREE(v))
  
```

Exercise 2.2

For the tree t shown in Figure 2.16, write down the vector v obtained by applying the function *IN_ORDER* to t . Then apply *GROW_TREE* to the vector v , and give the tree that you obtain. Is this the same as the original tree t ?

Exercise 2.3

Let v be the vector $[11, 22, 33, 44, 55]$. Give the tree t obtained by applying *GROW_TREE* to v . Then apply *IN_ORDER* to the tree t . Is the resulting vector the same as v ?

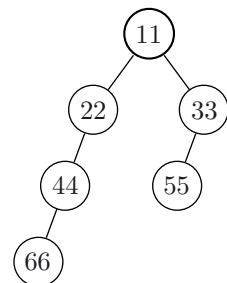


Figure 2.16

3 Sorted binary trees

In Unit 6, we noted that many tasks on sequences can be performed more efficiently if the sequence is sorted, meaning that the items appear in the sequence in accordance with some ordering of the data items. For example, [33, 2, 6, 2, 9, 8] is not sorted but [2, 2, 6, 8, 9, 33] is sorted. We noted that the way we find a word in a dictionary depends on the fact that the words are sorted within the dictionary.

Just as it can be useful to sort the data items within a sequence, it can similarly be useful to sort the data items in a binary tree. However, there are a variety of ways in which data within a tree may be sorted. In this section, we shall consider two of these. In one form of sorted tree, called a **binary search tree**, the data items appear so that they increase in size as one goes through the tree from left to right. We also consider **heap** trees, in which the data items increase in size as one works through the tree from top to bottom. For example, suppose that one needs to search a collection to see whether or not it contains a given item. In an unsorted sequence, one would need to check each item, say starting at the front and working through to the end. A binary search tree can be searched much more quickly.

To be able to sort items, we need to have an ordering relationship for the type of items that we are dealing with. Integers are ordered by their size, while characters are ordered by referring to their ASCII codes, and the data types **Int** and **Char** have corresponding comparison operations.

In a dictionary, strings are sorted in accordance with an ordering relationship called **lexicographic order**. In this, strings starting with ‘a’ are placed before strings starting with ‘b’, etc. This is not enough to order all strings, however. A string starting “aa” comes before one starting “ab”; a string starting “cor” comes before one starting “cos”, and so on. Let us describe this ordering of strings more carefully. We need a way of comparing two strings, say s and t , and deciding whether the proposition $s < t$ is **true** or **false**. We do this by comparing the strings character-by-character, working from the start of each string.

The proposition $s < t$ is **true** if one of the following conditions holds.

The first character of s is less than the first character of t .

The first characters of s and t are equal, and the second character of s is less than the second character of t .

The first characters of s and t are equal, the second characters of s and t are equal, and the third character of s is less than the third character of t . And so on.

To express this idea more formally and precisely, we need to deal with the general case, and to say what happens in any special cases. This is done below. For simplicity, we deal with two non-empty strings, s and t .

If the first characters of s and t are different, then the truth value of $s < t$ is the same as the truth value of $c < d$, where c is the first character of s and d is the first character of t .

If, for some positive integer n , the substring consisting of the first n characters of s is identical to the substring consisting of the first n characters of t , but the $n + 1$ st characters of the two strings are different, then the truth

These are not firm definitions of these ideas — just an informal indication of how these trees are arranged.

See Unit 6, Section 4.

The characters are compared using the operation $<$ of **Char**.

One could regard the empty string as smaller than all non-empty strings.

value of $s < t$ is the same as the truth value of $c < d$, where c is the $n + 1$ st character of s and d is the $n + 1$ st character of t .

If the string s is identical to the substring consisting of the first n characters of t (where n is the length of s), and t is longer than s , then $s < t$ is **true**.

If the strings s and t are identical then $s < t$ is **false**.

This description is not particularly easy to use. The procedure below is probably more user-friendly.

Strategy 3.1

Comparing strings

1. Compare the strings s and t character-by-character, starting at the front. Use the first characters where the strings are different to determine whether $s < t$ is **true**.
2. If s is identical to the front of t , and s is shorter than t , then $s < t$ is **true**.
3. If t is identical to the front of s , and t is shorter than s , or if t is identical to s , then $s < t$ is **false**.

A function *LESS_THAN* that compares two variables of type **mString** is specified below.

function *LESS_THAN*(s in **mString**, t in **mString**) **return in Bool**

pre s is not empty and t is not empty.

post The returned value is determined by Strategy 3.1 above.

The function *LESS_THAN* is available in the WorkPad.

The table below shows examples of various cases.

s	t	$s < t$	Comment
"abc"	"b"	true	Compare first characters: 'a' < 'b'.
"abc"	"ac"	true	First characters the same, compare second characters: 'b' < 'c'.
"abc"	"abd"	true	First two characters the same, compare third characters: 'c' < 'd'.
"abc"	"abcd"	true	s is identical to the start of t , and s is shorter than t .
"abc"	"ab"	false	t is identical to the front of s , and t is shorter than s .
"abc"	"abc"	false	The strings s and t are identical.

Activity 3.1

For each sequence below, give a sequence that contains the same items, and is sorted.

- (a) [55, 2, 78, 2, 4, 2, -1, 55] (from **Int**).
- (b) ['z', 'a', 'Z', 'A', '2', '1'] (from **Char**).
- (c) ["zebra", "Zebra", "jon", "jonathon"] (from **mString**).

Discussion We use the comparisons described above for each type of item. Remember that characters are compared by comparing their ASCII codes, and in particular that any upper-case letter is smaller than any lower-case letter. So, for example, ‘Z’ < ‘a’ is **true**. This also affects the ordering of strings, since we compare the two strings character-by-character, using the comparison < on **Char**. So, for example, “Zebra” < “jon” is **true**, because ‘Z’ < ‘j’ is **true**. Note also, that if a sequence contains more than one copy of an item, the sorted version should contain the same number of copies. (So, for example, the sequence in (a) contains three copies of 2.)

- (a) [-1, 2, 2, 2, 4, 55, 55, 78]
- (b) ['1', '2', 'A', 'Z', 'a', 'z']
- (c) ["Zebra", "jon", "jonathon", "zebra"]

There are circumstances where one might want to use a different ordering of characters (and of strings), but in this course we shall keep to the simple ordering based directly on the ASCII codes.

3.1 Binary search trees

The first type of sorted tree that we shall look at is a binary search tree. This may be regarded as one of the most useful data structures invented! There are numerous algorithms for manipulating such trees.

A binary search tree is one where, at every node in the tree, the value of the element at this node is greater than every node in the subtree below it and to the left and is less than every node in the subtree below it and to the right. This has to be true not just for the root node, but for every non-leaf node in the tree. We can give a formal definition as follows. A binary tree is a **binary search tree** if:

- it is an empty tree; or
- the value at the root is greater than (or equal to) every value in its left subtree, and is less than (or equal to) every value in its right subtree, and its left subtree and its right subtree are themselves both binary search trees.

This definition is recursive. It is easy to express, but rather difficult to use directly, since it requires us to examine every subtree of the given tree.

For convenience, we may shorten “binary search tree” to BST. One way of testing a given tree t to see whether or not it is a BST is to examine the sequence $IN_ORDER(t)$ to see whether it is sorted. We have the following.

If $IN_ORDER(t)$ is a sorted sequence then the tree t is a BST.

If the tree t is a BST then $IN_ORDER(t)$ is a sorted sequence.

Activity 3.2

Which of the trees in Figure 3.1 are binary search trees?

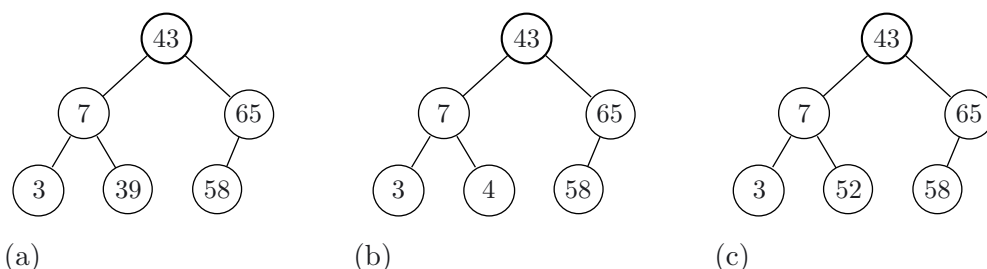


Figure 3.1

Discussion Only the tree in (a) is a BST. The tree in (b) is not a BST because the node 7 has 4 as a right child, and $7 \leq 4$ is **false**. The tree in (c) is not a BST because the root is 43, and the left subtree contains 52 at a node, and $52 \leq 43$ is **false**. The sequences $IN_ORDER(t)$ for each of these trees t are given below.

- (a) [3, 7, 39, 43, 58, 65]. This is sorted.
 (b) [3, 7, 4, 43, 58, 65]. This is not sorted, since $7 \leq 4$ is **false**.
 (c) [3, 7, 52, 43, 58, 65]. This is not sorted, since $52 \leq 43$ is **false**.

The function $GROW_TREE$, discussed in Section 2, turns a sequence into a tree in a way that reflects the order of the items within the sequence in the “left to right” structure of the tree. This means that if we apply $GROW_TREE$ to a sequence that is sorted, then we will obtain a BST. Indeed, we have the following result.

If v is a sorted sequence then $GROW_TREE(v)$ is a BST.
 If $GROW_TREE(v)$ is a BST then v is a sorted sequence.

As the name suggests, a binary search tree is a structure suited to searching to see whether it contains a particular element. Such searches can be carried out quickly in large collections. We illustrate this process with an example.

EXAMPLE 3.1

Figure 3.2 shows a BST containing the following strings: “bison”, “earwig”, “zebra”, “hawk”, “terrapiin”, “ant”, “turtle”.

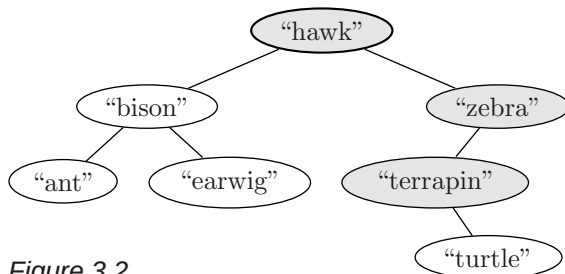


Figure 3.2

To search for the string “tapir” in the tree we proceed as follows.

1. Start at the root node and compare “tapir” with the string there.
2. The string “tapir” is not equal to “hawk”. What is more, in the lexicographic order, “tapir” > “hawk”. So if “tapir” is in the tree it cannot be in the left subtree. Consequently, we need only compare “tapir” with nodes in the right subtree.
3. So we next compare “tapir” with the root of the right subtree, which is “zebra”. Again, the strings “tapir” and “zebra” are not equal. This time, “tapir” < “zebra”. So we need only look in the left subtree below this node.
4. The root of this tree is “terrapiin”. We find that “tapir” is not equal to “terrapiin”, and that “tapir” < “terrapiin”. So we look at the left subtree below “terrapiin”. This is empty, so the search can stop.

The shaded nodes are those examined when searching this tree for “tapir”.

Strings are compared using the lexicographic order described above.

We can now be sure that “tapir” does not appear in the tree shown in Figure 3.2. By exploiting the fact that this is a BST, we have completed the search by comparing “tapir” with just three of the seven items stored in the tree.

To search for an item in this way in a BST, the number of items we need to examine is at most the depth of the tree. For a large collection, this can compare very favourably with searching through the collection item-by-item, where the maximum number of comparisons that may be needed is the size of the collection. For example, if a collection of a million items is organised in a complete BST, then a search will require examination of at most 20 items. Checking each item in turn may require us to examine each of the million items.

See page 21.

Suppose that we have a collection of items (say the dictionary for a spell checker) organised in a BST. We may want to add items to this (perhaps technical terms not in the dictionary). We want to add new items in a way that preserves the structure of the binary search tree. We can do this using a procedure quite similar to the way in which we search in a BST. We move down the tree, as if we were searching for the item that we wish to insert, until we meet an empty node. This will be the location that we will use to store the new element. For example, if we want to add “tapir” to the tree in Figure 3.2, it would go below and to the left of “terrapiin”.

Activity 3.3

Starting with the tree in Figure 3.2, determine the trees obtained by adding first “kiwi”, then “iguana” using the procedure described above.

Discussion The tree obtained by adding “kiwi” is shown in Figure 3.3(a).

To determine where this should go, start at the root of the tree in Figure 3.2.

Compare “kiwi” with “hawk”. We have “kiwi” > “hawk”.

So move down the tree to the right.

Compare “kiwi” with “zebra”. We have “kiwi” < “zebra”.

So move down the tree to the left.

Compare “kiwi” with “terrapiin”. We have

“kiwi” < “terrapiin”. So move down the tree to the left.

We now find an empty subtree. This is the place to store “kiwi” at a new node.

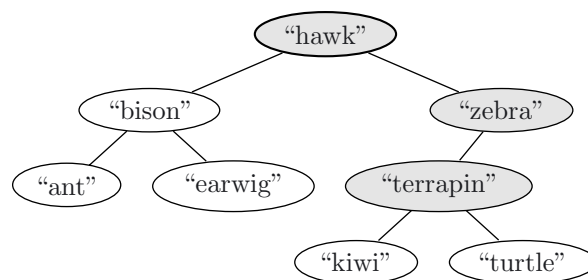


Figure 3.3 (a)

Figure 3.3(b) shows the tree obtained by adding “iguana”. To determine where this should go, start at the root of the tree in Figure 3.3(a).

Compare “iguana” with “hawk”. We have

“iguana” > “hawk”. So move down the tree to the right.

Compare “iguana” with “zebra”. We have

“iguana” < “zebra”. So move down the tree to the left.

Compare “iguana” with “terrapiin”. We have

“iguana” < “terrapiin”. So move down the tree to the left.

Compare “iguana” with “kiwi”. We have

“iguana” < “kiwi”. So move down the tree to the left.

We now find an empty subtree. This is the place to store “iguana” at a new node.

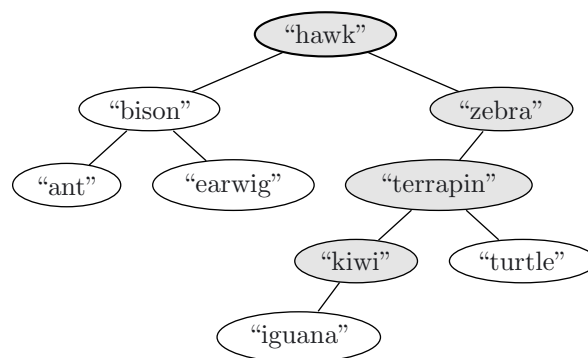


Figure 3.3 (b)

The procedure for adding an item to a BST, and a corresponding function, are given below.

Strategy 3.2**Adding an element to a BST**

To add *elem* to a BST *t*, proceed as follows. Move down through the tree *t* until we reach an empty subtree, moving as follows.

1. Compare *elem* to the root of *t*. If $elem \geq$ the root of *t*, then move down and right. Otherwise, move down and left.
2. Compare *elem* with the current node of *t*, say *node*. If $elem \geq node$, move down and right. Otherwise, move down and left.
3. Continue moving down until you come to an empty subtree. This is the place to add *elem* to *t*.

Our BSTs can contain more than one copy of the same item, and this strategy works on that basis. In some circumstances, one would not want to permit multiple copies, and in such a case a modification of this strategy for insertion would be required.

function *INSERT_TREE*(*t* in BinTree of X, *elem* in X) **return** in BinTree of X

pre *t* is a binary search tree.

post The returned value is a BST in which the item *elem* has been added to *t*. The place at which *elem* is added is determined by Strategy 3.2.

The function *INSERT_TREE* is available in the WorkPad.

We can see from Figure 3.3(b) that Strategy 3.2 can lead to a tree that is not balanced. In such a tree, the efficiency of searching will be decreased. If a large number of items are added, the tree may become very unbalanced, with a significant loss of efficiency. In such a case, it would be useful to rearrange the tree so as to make it balanced (while maintaining its structure as a BST). This can be done using functions described in Section 2. Firstly, apply *IN_ORDER* to the tree. This will give a vector, and this vector will be sorted, since we started with a BST. Then apply *GROW_TREE*. This will produce a tree that is balanced, and is a BST (because the vector is sorted).

If we require a balanced tree after every insertion, we would need to add an element to the tree in a more complicated way. A simpler approach is to update the tree using the functions *IN_ORDER* and *GROW_TREE*, but to only carry out these balancing operations occasionally.

If we have a vector that is not sorted, and want to put the items into a BST, we could start with an empty tree and add the items one at a time, using *INSERT_TREE*. Or we could sort the vector first, then apply *GROW_TREE* to the sorted vector. This second approach will always yield a balanced tree. The function specified below is available in the WorkPad.

function *SEARCH_TREE* (*v* in Vector of X) **return** in BinTree of X

pre **true**.

post The returned value is the tree obtained by first sorting *v*, then applying *GROW_TREE* to the resulting vector.

Note that *GROW_TREE* does *not* change the state of the input vector *v*.

Activity 3.4

Draw trees showing the state of the variable t after each of statements (1), (2), (3) and (4) in the code fragment below.

```
var v in Vector of Int
var t in BinTree of Int
v.setVector(11,33,44,22,66,55,77)
t <-- SEARCH_TREE(v)           //(1)
t <-- INSERT_TREE(t, 34)       //(2)
t <-- INSERT_TREE(t, 35)       //(3)
v <-- IN_ORDER(t)
t <-- GROW_TREE(v)            //(4)
```

Discussion The states are shown in Figure 3.4.

The vector v is set to [11, 33, 44, 22, 66, 55, 77]. To form $SEARCH_TREE(v)$, sort the vector v , to get [11, 22, 33, 44, 55, 66, 77]. Then apply $GROW_TREE$ to this sorted vector. That gives the tree in Figure 3.4(a). (Note that the state of v is not changed by statement (1).)

To add 34 to the tree, $INSERT_TREE$ will use Strategy 3.2. Look at the tree in Figure 3.4(a). We compare the element to be added, 34, with elements of that tree as follows.

First the root: $34 < 44$, so go down and left;
 then 22: $34 > 22$, so go down and right;
 then 33: $34 > 33$, so go down and right.

We now come to an empty subtree, so this is where 34 is added, to give the tree shown in Figure 3.4(b).

To add 35 to the tree in Figure 3.4(b), compare 35 with elements of that tree as follows.

First the root: $35 < 44$, so go down and left;
 then 22: $35 > 22$, so go down and right;
 then 33: $35 > 33$, so go down and right;
 then 34: $35 > 34$, so go down and right.

We now come to an empty subtree, so this is where 35 is added, to give the tree shown in Figure 3.4(c).

Applying IN_ORDER to the tree in Figure 3.4(c) gives the vector [11, 22, 33, 34, 35, 44, 55, 66, 77]. (Since the tree is a BST, we can be sure that we will get a sorted vector from IN_ORDER here.)

Applying $GROW_TREE$ to this vector gives the tree in Figure 3.4(d). (Since the vector was sorted, this is bound to be a BST. Since it is a value returned by $GROW_TREE$, this tree is bound to be balanced.) The last two instructions rearrange the items in the BST in Figure 3.4(c) to obtain a BST that is balanced.

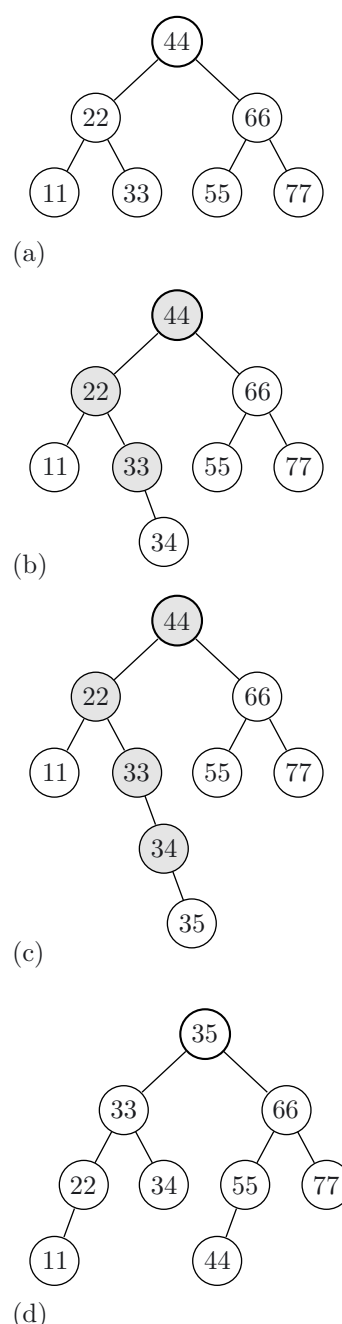


Figure 3.4

3.2 Heap trees

We now consider a different form of sorted binary tree, called a **heap tree** (which we may shorten to just “heap”). A heap is a complete tree. Now in a BST, smaller values are kept *to the left* of larger values. In a heap tree, smaller values are stored *above* larger values. This ensures that the smallest item in a heap is at the root of the tree. To determine whether a tree is a heap we need to check that each node (other than the root) is greater than (or equal to) its parent, and that the tree is complete.

For example, the tree in Figure 3.5 is a heap. We can verify that as follows.

First check that the tree is complete — it is.

Look at the children of the root node: we have $3 \leq 9$ and $3 \leq 21$.

In the right subtree: $21 \leq 37$ and $21 \leq 70$.

In the left subtree: $9 \leq 24$ and $9 \leq 31$.

Going “left and left”: $24 \leq 46$ and $24 \leq 71$.

Going “left and right”: $31 \leq 77$.

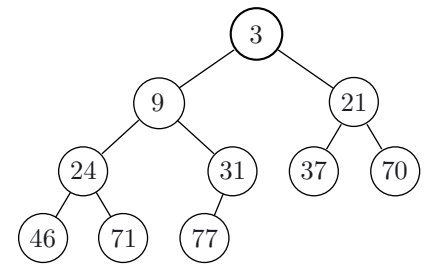


Figure 3.5

A recursive characterisation can be used to give a more formal definition of a heap, as below. A binary tree is a **heap tree** if it is a complete tree, and:

- it is an empty tree; or
- the value at the root is less than (or equal to) both its left and right children (if they exist), and its left subtree and its right subtree are themselves both heap trees.

In a heap, there can be values at lower levels that are smaller than values at higher levels, as is illustrated in Figure 3.5 (where 46 is smaller than 70). This can occur where the nodes are not connected by branches. Nor is there any significance in the way values appear at nodes on the same level. Smaller values can appear to the left or to the right. The only condition that must hold in a heap is that the value stored at a child node is greater than (or equal to) the value at its parent.

Activity 3.5

Which of the trees in Figure 3.6 are heap trees?

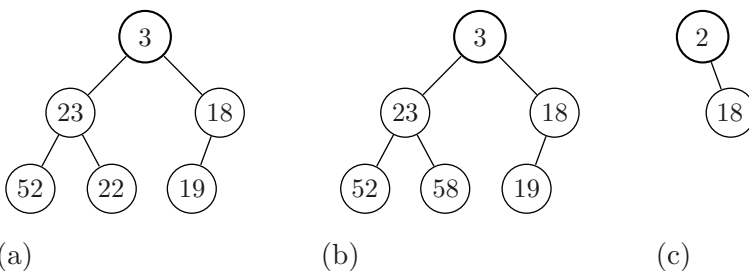


Figure 3.6

Discussion

- The tree in Figure 3.6(a) is not a heap tree. In the left subtree, 22 appears as a child of 23, and $23 \leq 22$ is **false**.
- The tree in Figure 3.6(b) is a heap tree.
- This tree is not a heap tree, because it is not a complete tree.

We saw that BSTs hold data in a way that is convenient for quick searching. Heap trees are useful in different contexts. One important application of a heap is to model a **priority queue**.

We need to distinguish a priority queue from the most common form of queue, like that at a supermarket checkout. Such queues are “first in first out”: shoppers join at one end of the queue (the back) and leave from the other end (the front). Such a queue is naturally seen as a sequence.

You may see the acronym FIFO used for such queues.

A priority queue is one in which each item in the queue has an associated priority. Items with higher priority will be given precedence in the queue, and so may leave it before items that joined the queue before them. For example, a company’s IT support department might give each new task a priority code as it arrives, say: 1 for “urgent”; 2 for “pressing”; 3 for “routine”; 4 for “whenever possible”. In our examples, the smaller the integer, the higher the priority it represents.

There are two fundamental processes associated with any queue — a new item joining the queue, and an item leaving the queue. For a sequential (FIFO) queue, these processes are quite simple. For a heap tree acting as a priority queue, neither is straightforward.

When an item is removed from the queue, we need to reorganise the remaining items so that we still have a heap. For example, the tree in Figure 3.7 is a heap. If we view this as a priority queue, then one thing is clear: the next item to be “dealt with” — and so to leave the queue — is the item at the root of the tree. Now if we simply delete the root from the heap in Figure 3.7, we do not even have a tree!

To model such a situation fully, we would need a more complicated form of data than just a single integer, since we would also need some way of identifying the task with which this priority number is associated — but we do not want to discuss such issues here. Just think of the numbers in the heap tree as representing some form of priority, with smaller numbers having the higher priority.

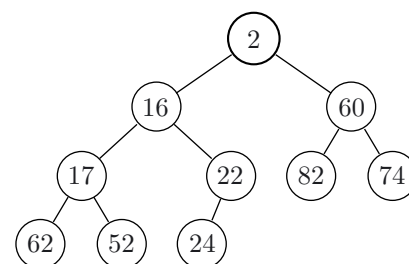
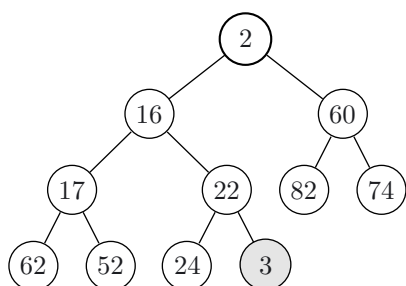


Figure 3.7

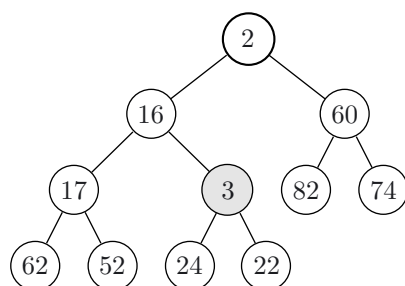
Adding an item to the queue also raises problems. For example, in the tree in Figure 3.7, there is only one place where an item can be added so that we still have a complete tree — which is below and to the right of 22. However, suppose that the next item to be added is 3. If we put that below and to the right of 22, we will no longer have a heap. After adding 3 to the tree, we would then need to reorganise the items in the tree to retrieve the heap structure.

We shall look first at the process of adding a new item to a heap. Figure 3.8(a) shows the tree that we obtain when the item 3 is added to the heap tree in Figure 3.7. It is added below and to the right of 22. Adding the new item there ensures that we have a complete tree — but the tree is not a heap, since 3 is a child of 22, but 3 is smaller than 22.

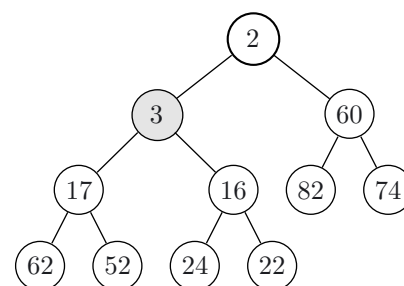
To restore the heap property, the value 3 has to be moved upwards in the tree until it reaches a suitable position relative to the other items in the tree. This is done by comparing the new item with its parent, and exchanging these two items if necessary.



(a)



(b)



(c)

Figure 3.8

In Figure 3.8(a), the new item 3 is smaller than its parent 22; so we exchange the positions of these two items in the tree, giving the tree in Figure 3.8(b).

We now compare 3 with its new parent, 16. Again 3 is smaller than 16, so we need to exchange these two items. This gives the tree in Figure 3.8(c).

Now compare 3 with its new parent, 2. These items are correctly sorted: 3 is larger than 2. So no more exchanges are needed.

The strategy for adding an item to a heap is laid out below.

Strategy 3.3

Adding an item to a heap

To add a new item x to a heap tree, proceed as follows.

1. Add x to the heap at the first available place that keeps a complete tree (x will now be at a leaf node).
2. If the new item x is less than the item at its parent node then exchange these items.
3. Repeat the process of comparing the new item with the current parent node and exchanging if necessary.
4. Stop when the new item is greater than (or equal to) its parent.

This procedure is bound to result in a heap.

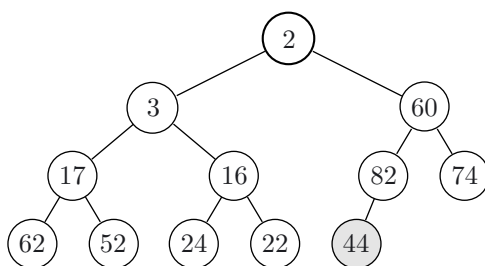
This shows why it can be useful to use a heap rather than a sequence as a priority queue. The number of steps required to move the new item to a suitable place in the heap is at most the depth of the tree. If the priority queue were stored as a sequence, and the new item was moved through the sequence to its appropriate place by a similar sequence of exchanges, then the number of steps required could be as much as the length of the sequence. For large collections, we get a similar improvement in efficiency to that achieved by searching in a BST rather than in an unsorted sequence.

It may take a little thought to see this.

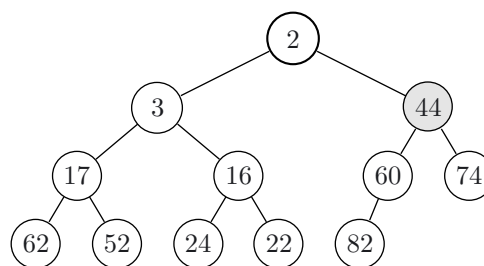
Activity 3.6

Determine the heap tree obtained when Strategy 3.3 is used to add the item 44 to the heap shown in Figure 3.8(c).

Discussion We obtain the heap shown in Figure 3.9(b). The new item 44 is added below and to the left of 82 (see Figure 3.9(a)). It is first compared with 82, and 44 and 82 are exchanged (since $44 < 82$ is **true**). It is then compared with 60, and 60 and 44 are exchanged. 44 is then compared with 2, and is not exchanged, since $44 \geq 2$ is **true**.



(a)



(b)

Figure 3.9

We now look at removing an element from a heap. If the heap is regarded as a priority queue, then the only item that can be removed is that at “the front of the queue”, which is the root node in the tree. When the root item is removed from the heap, we will need to reorganise the remaining items in a suitable way, so that the heap property is restored. We will describe a procedure that does this in an efficient fashion.

The strategy is first to move the element at the “last leaf” to the root. The resulting tree will not be a heap, since the new root came from the bottom of the tree, and so is likely to be larger than elements below it. We then move this item from the root node down the tree, until we once more have a heap. This is again achieved by a series of exchanges between this “sinking item” and one of its children. In general, there are two possible ways to pass the sinking item down the tree. The first step is to determine the smaller value of the two children of the sinking item. If this smaller value is less than the sinking item, then we make an exchange. We repeat this process until the sinking item can “sink” no further—either because it is smaller than both of its children or because it has no children. The tree will then be a heap again.

The procedure for removing the root element from a heap is given below.

Strategy 3.4

Removing the root item from a heap

1. Delete the old root.
2. Place the value of the “last leaf” node at the root. This is referred to as the “sinking item”.
3. Remove the last leaf position from the tree.
4. Starting with the sinking item at the root node, repeat the following steps.
 - 4.1 Determine the smallest child of the current position of the sinking item.
 - 4.2 If this smallest child is smaller than the sinking item, then exchange it with the sinking item.
5. The repetition stops when the sinking item is smaller than (or equal to) all its children (or it has no children).

At step 4, if the children are equal and are smaller than the sinking item, then exchange the sinking item with the left child.

This procedure is bound to result in a heap. It involves at most $d - 1$ exchanges, where d is the depth of the tree.

The process is best seen through an example.

EXAMPLE 3.2

Figure 3.10 shows a heap. We will follow the above strategy to remove the root value and adjust the tree.

The “last leaf” stores the item 22. The first step is to remove the current root item, 2, and replace this with the item 22 from the “last leaf”. (This “last leaf” position disappears from the tree.) This gives the tree in Figure 3.11(a).

We now need to “sink” 22 down the tree as far as is needed. First, find the smaller of its children. These are 3 and 60, and 3 is the smaller of these. Then compare 22 with 3. Since 22 is larger than 3, we exchange these two items, to obtain the tree in Figure 3.11(b).

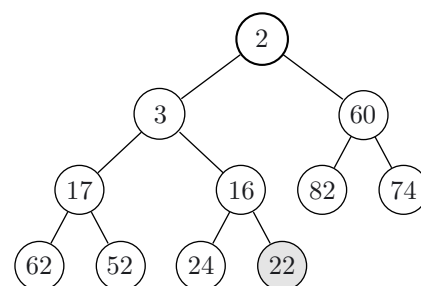


Figure 3.10

In this new tree, determine the smaller child of the “sinking item”, 22. Its children are 17 and 16, and 16 is the smaller of these. Now compare 22 with 16. Since 22 is larger than 16, exchange these items. This gives the tree in Figure 3.11(c).

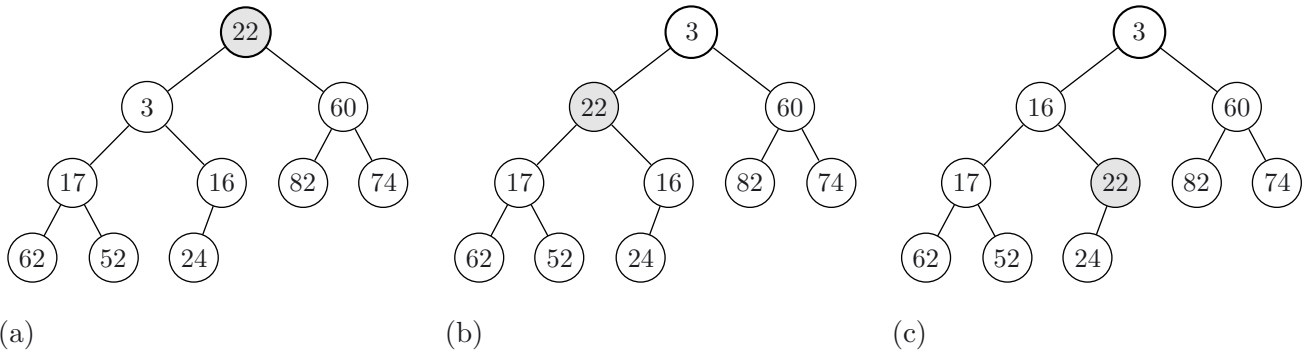


Figure 3.11

Now, the sinking item has only one child. This is 24. Since 22 is smaller than 24, we do not make an exchange, and the process is complete. The tree in Figure 3.11(c) is again a heap — that formed by removing the root from the heap in Figure 3.10.

One might arrange the items in a tree such that the largest item is at the root, and items get smaller as one goes down the levels of the tree. This is a “mirror image” of the way we have defined a heap, and effectively the same procedures can be used to maintain the structure of such a tree. If you read other texts, you will find that trees sorted in this way are also regarded as heap trees. For simplicity, in this course we shall confine use of the term “heap” to trees sorted with the smaller items above.

In Unit 6 we discussed an algorithm (bubble sort) for sorting a vector. We can use the processes on a heap as the basis for an alternative sorting algorithm (called **heap sort**). Starting with a vector, use Strategy 3.3 to add the items from the vector to a heap, one-by-one. Then use Strategy 3.4 to remove the items one by one from the heap, and put them back into a vector. This will sort the items in the vector. At first glance, this approach might seem likely to be slower than bubble sort, but for a large collection it is, in fact, much quicker.

The second step in this process is illustrated in Activity 3.7.

Activity 3.7

Let v be of type **Vector of Int**, and t be of type **BinTree of Int**, where v is empty, and t has the state shown in Figure 3.12. (Note that the tree in Figure 3.12 is a heap.)

Suppose that we repeat the following steps until t is empty.

1. Add the root element of t to the end of v .
2. Remove the root element from t using Strategy 3.4, and change the state of t to become this new heap.

Write down the final state of the vector v .

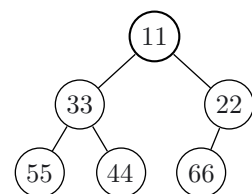
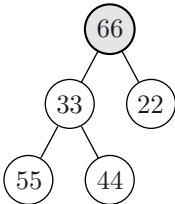
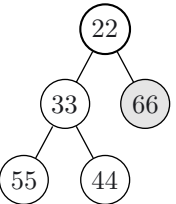
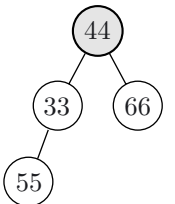
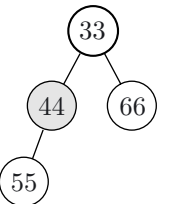
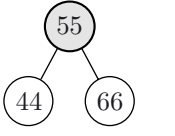
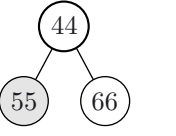
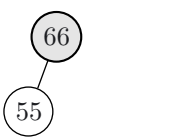
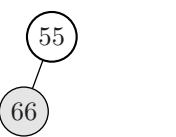


Figure 3.12

Discussion The states of the variables v and t after each execution of steps 1 and 2 are tabulated below. (Note that each state given below for t is a heap tree.)

v	Strategy 3.4	Replace root	t
[11]	<i>First execution</i> Remove 11. Put 66 at root. $22 < 33$. $66 > 22$, so exchange.		
[11, 22]	<i>Second execution</i> Remove 22. Put 44 at root. $33 < 66$. $44 > 33$, so exchange. $44 < 55$, so stop.		
[11, 22, 33]	<i>Third execution</i> Remove 33. Put 55 at root. $44 < 66$. $54 > 44$, so exchange.		
[11, 22, 33, 44]	<i>Fourth execution</i> Remove 44. Put 66 at root. $66 > 55$, so exchange.		
[11, 22, 33, 44, 55]	<i>Fifth execution</i> Remove 55. Put 66 at root.		
[11, 22, 33, 44, 55, 66]	<i>Sixth execution</i> Remove 66.		tree is empty.

Objectives for Section 3

After studying this section you should be able to do the following.

- ☐ Recognise and use the terminology: lexicographic order (of strings); binary search tree (or BST); heap (tree).
- ☐ Given two strings, determine which comes first in the lexicographic ordering.
- ☐ Determine whether a given tree is a BST.
- ☐ Determine whether a given tree is a heap.
- ☐ Understand how to rebalance a BST.
- ☐ For given inputs, determine the value returned by each of the functions: *LESS_THAN*; *INSERT_TREE*; *SEARCH_TREE*.
- ☐ Given a suitable tree, determine the outcome of various procedures:
 - to determine whether a given item appears in a BST;
 - to add an item to a BST;
 - to add an item to a heap;
 - to delete the item at the root of a heap.

Exercises on Section 3

Exercise 3.1

Suppose that v is of type **Vector of Int**, t is of type **BinTree of Int**, and t is in the state shown in Figure 3.13. Determine the states of v and t after each statement in the code fragment below is executed.

```
v <-- IN_ORDER(t)
t <-- SEARCH_TREE(v)
v <-- IN_ORDER(t)
t <-- INSERT_TREE(t, 30)
```

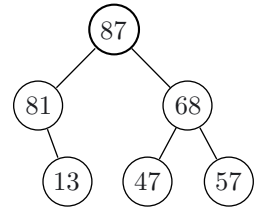


Figure 3.13

Exercise 3.2

For the heap tree shown in Figure 3.7, determine the tree obtained when the root is removed using Strategy 3.4.

Exercise 3.3

- Suppose that v is a sorted vector. Will $COMPLETE_TREE(v)$ necessarily be a heap tree?
- If t is a heap, will $LEVEL_ORDER(t)$ necessarily be a sorted vector? (*Hint*: consider the tree in Figure 3.12, for example.)

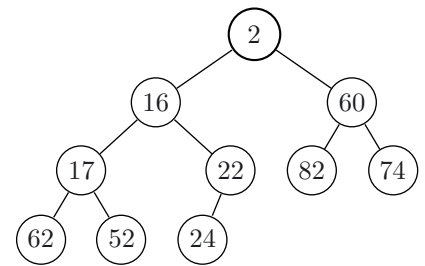


Figure 3.7

Summary of Unit 8

- A binary tree is a highly structured collection of data items. A non-empty binary tree has a root (which holds the only item that is readily accessed), and left and right subtrees that are themselves binary trees. The structure of a tree is conveniently shown in a diagram, and we show the nodes of a tree at a series of levels. The level of a node is determined by the number of branches there are between the node and the root of the tree.
- A tree of depth n can hold $2^n - 1$ items. To fully exploit its capacity for storage, a tree should be balanced, or, even better, complete.

Ways of moving data between a binary tree and a sequence were discussed. The function $COMPLETE_TREE$ places data items from a vector into a complete tree in the following way. The first item in the vector is placed at the root of the tree; the next two items are placed at level 2 in the tree; the next four items at level 3 in the tree, and so on. Each level is filled from left to right. The function $LEVEL_ORDER$ reads items from a tree into a vector in a way that is the reverse of $COMPLETE_TREE$.

The function $GROW_TREE$ places items from a sequence into a balanced tree in a way that preserves the “left to right” order of items in the sequence. The function IN_ORDER reads items from a tree into a vector in a way that is the reverse of $GROW_TREE$.

3. Many common forms of data, such as integers, characters and strings, have an intrinsic ordering, which can be exploited in code if there is a corresponding type function or method for comparing two items. If items are placed in a tree in a way that is compatible with such an ordering of the data items, we call the tree “sorted”. We considered two forms of sorted tree: binary search trees (BSTs) and heap trees. In a BST, smaller items appear “to the left” of larger items. In a heap, smaller items appear “above” larger items (that is, the item at any node must be greater than or equal to the value at its parent node). Procedures for executing various operations on such trees were given: searching for an item in a BST; adding an item to a BST; adding an item to a heap; deleting the root item from a heap.

Solutions to the exercises

Solution 1.1

- (a) The returned values are: (i) 4; (ii) 36; (iii) 81; (iv) **true**.
 (b) The final state of t is as in Figure S1. Notice that the effect of the code is to exchange the right and left subtrees of t .

Solution 1.2

An implementation is given below. The strategy used is to:
 declare a temporary tree variable;
 then set the root to be the root of the input tree;
 then set the left subtree to be the left subtree of the input tree.

```
function DELETERIGHT(t)
{
  var outTree in BinTree of X
  outTree.setRoot(t.getRoot())
  outTree.setLeftTree(t.leftTree())
  return outTree
}
```

You may have included a statement `outTree.setRightTree(ET())` to set the right subtree to be empty. There is nothing wrong in doing that, but it is not necessary, since the right subtree of `outTree` will already be empty after the statement `outTree.setRoot(t.getRoot())`.

Alternatively, one can set `outTree` using the single statement

```
outTree.setTree(t.getRoot(), t.leftTree(), ET())
```

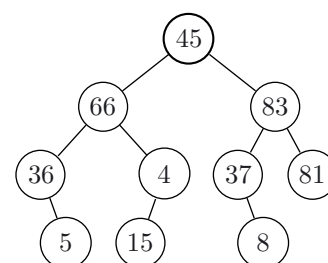


Figure S1

Solution 1.3

An implementation is given below.

```
function LEFTLEAF(t)
{
  var temp in BinTree of X
  temp <-- t.clone()
  while (NOT(temp.leftTree().isEmpty()) \ /
        NOT(temp.rightTree().isEmpty()))
  {
    if (temp.leftTree().isEmpty()) then
    {
      temp <-- temp.rightTree()
    }
    else
    {
      temp <-- temp.leftTree()
    }
  }
  return temp.getRoot()
}
```

Solution 2.1

The final state of t is as shown in Figure S2.

Each of *COMPLETE_TREE* and *GROW_TREE* returns a balanced tree, and when the functions are applied to the same vector the resulting trees will have the same depth. So we would expect that the tree resulting from this code would be balanced — and we can see from the figure that it is. The tree is not complete, though; the bottom level has not been filled from the left.

Solution 2.2

We obtain the vector [66, 44, 22, 11, 55, 33]. Applying *GROW_TREE* to this gives the tree shown in Figure S3. This is not the same as the original tree t . This could be anticipated, since the tree in Figure 2.16 is not balanced.

Solution 2.3

We obtain the tree shown in Figure S4. Applying *IN_ORDER* to that tree gives the vector [11, 22, 33, 44, 55]. This is the same as v — and this could have been anticipated, since applying *GROW_TREE* and then *IN_ORDER* will always take us back to the vector we started with.

Solution 3.1

The states are as tabulated below. *SEARCH_TREE* first sorts the items in the vector (without changing the state of v) before placing them into a tree using *GROW_TREE*, so giving a BST containing the same items as the original tree in Figure 3.13.

Statement	v	t
$v \leftarrow \text{IN_ORDER}(t)$	[81, 13, 87, 47, 68, 57]	Figure 3.13
$t \leftarrow \text{SEARCH_TREE}(v)$	[81, 13, 87, 47, 68, 57]	Figure S5(a)
$v \leftarrow \text{IN_ORDER}(t)$	[13, 47, 57, 68, 81, 87]	Figure S5(a)
$t \leftarrow \text{INSERT_TREE}(t, 30)$	[13, 47, 57, 68, 81, 87]	Figure S5(b)

The way *INSERT_TREE* places 30 into the tree in Figure S5(a) is as follows.

30 < 68, so go left.
 30 < 47, so go left.
 30 > 13, so go right.

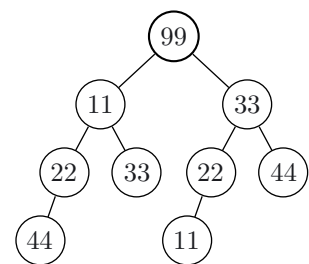


Figure S2

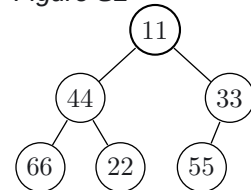


Figure S3

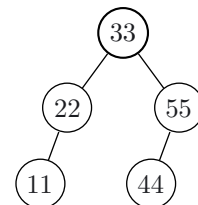


Figure S4

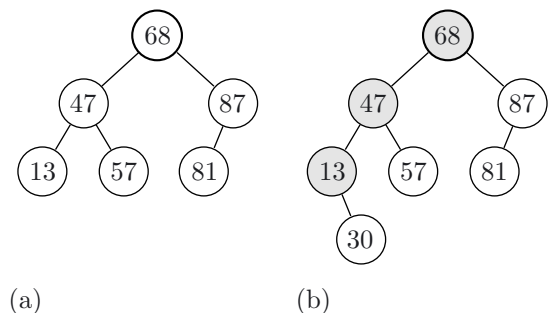


Figure S5

Solution 3.2

The tree is shown in Figure S6(c). The steps in executing Strategy 3.4 are given below (starting with the tree in Figure 3.7).

Delete the root item.

Move the item from the “last node”, which is 24, to the root. (See Figure S6(a).)

The smallest child of the root is 16 (since $16 < 60$).

$24 > 16$, so exchange 24 with 16. (See Figure S6(b).)

The smallest child of the “sinking item” is now 17 (since $17 < 22$).

$24 > 17$, so exchange 24 and 17. (See Figure S6(c).)

The smallest child of the sinking item is now 52 (since $52 < 62$).

$24 < 52$, so there are no further exchanges.

Solution 3.3

(a) Yes. *COMPLETE_TREE*(v) will certainly be a complete tree. Items from v will be placed into *COMPLETE_TREE*(v) starting at the root and working down through the levels (and filling each level from left to right). Since v is sorted, the smallest item in v is placed at the root. The next smallest item will be put at the left child of the root, the next at the right child of the root; then we move to level 3, and so on. Any item at level n will appear later in the vector v than any item at level $n - 1$, and so, since v is sorted, the items at level n will all be larger than those at level $n - 1$. This ensures that, as one moves down branches in the tree, items will get larger—in the way required in a heap.

(b) No. The tree in Figure 3.12 is a heap. Applying *LEVEL_ORDER* to that tree will give the vector [11, 33, 22, 55, 44, 66]. This is not sorted. The size of the items in a heap increases as you go down branches, but it does not need to increase as you read each level from left to right. Consequently, *LEVEL_ORDER* applied to a heap tree does not necessarily give a sorted vector.

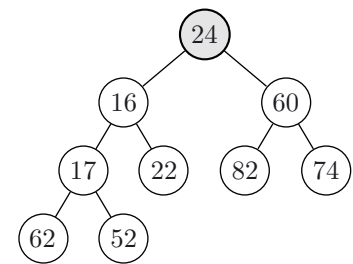


Figure S6 (a)

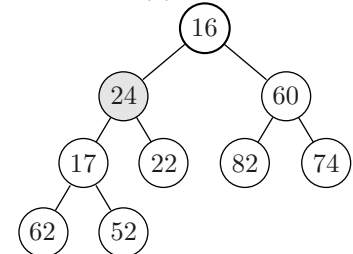


Figure S6 (b)

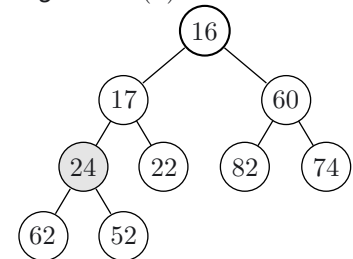


Figure S6 (c)

Index

balanced (tree), 20
 binary search tree, 34
 binary tree, 5
 linear notation for a, 10
 balanced, 20
 complete, 22
 left branch in a, 6
 left subtree of a, 7
 right branch in a, 6
 right subtree of a, 7
 root of a, 6
 branch, 6
 BST, 34
 child node, 6
 complete tree, 22
 depth, 7
 empty tree, 6

FIFO queue, 40
 heap (tree), 39
 heap sort, 43
 in-order traversal, 25
 interior node, 6
 leaf (node), 6
 left child, 7
 left subtree, 7
 level (in a tree), 7
 completely filled, 21
 level order traversal, 24
 lexicographic order, 32
 node, 6
 pancaking (a tree), 26
 parent node, 6

priority queue, 40
 right child, 7
 right subtree, 7
 root, 6
 subtree, 8
 tree, 4
 binary, 5
 branch of a, 5
 depth of a, 7
 level in a, 7
 node of a, 5
 subtree of a, 8
 tree traversal, 20
 breadth first, 24
 in-order, 25
 level order, 24
 tree walk diagram, 26